



**DATA MODELLING, MANAGEMENT AND GOVERNANCE
(CIS108-6)**

**A Data Management System Design Report for Green
Futures'**

ENoch ADJEI ADJETEY

**MSC APPLIED COMPUTING AND INFORMATION
TECHNOLOGY**

DECLARATION

I do confirm that to the best of my knowledge, this report is entirely original work intended to earn an exceptional assessment score in Unit Code CIS108-6, University of Bedfordshire, and that it contains no academic dishonesty or plagiarism that has been previously approved for any evaluation at any university except where due acknowledgement has been made in the work.

EXECUTIVE SUMMARY

In this report, I have given a detailed insight on how I designed and developed a systematic record management system for Green Future. The system will have functionalities such as maintain and retrieve information about members project activity, manage members project booking, and check project availability. The system design will provide administrative staff with access to track member participation in project activities and manage project schedules in order to monitor and manage system information. The standard design and development of this system should meet objectives which include enabling members to join the organization, view and update their profiles, whilst allowing administrative staff to create, query, modify, and delete members, staff and project information. The tools used to develop the system include, but are not limited to virtual paradigm for designing use case, activity diagram and entity relationship diagrams (ERDs). The ERD will play a crucial role in database design by providing a visual representation of the database schema, facilitating communication, aiding in planning and design, ensuring data integrity, and serving as blueprint for implementing the Green Future organization. In addition, SQL statements was used to showcase the system's functionality by displaying current project registrations by a member, searching for projects led by a specific staff member and displaying the most active members.

Chapter 1

1.0 Introduction

Data has become an essential component to most organizations and firms, playing a crucial role in decision making which is key to the growth of any firm. In recent years, there has been a significant increase in the demand for data storage due to the fragility and complexity it brings to organizations and businesses. To keep up with the rapidly evolving technology landscape, organizations have upgraded their systems to effectively and efficiently secure, manage, recover, store, investigate, and test the integrity of their data management systems. This ability to manipulate data is an important aspect of any business. (Silver et al. 1995) data is useful when it can be processed to information.

Over time, as failures were analyzed and concepts were refined, new technologies emerged. This led to the development of peer-to-peer (P2P) systems, data integration, database clusters, web and XML data management systems, stream data management, and cloud data management. There has been a rapid acceleration in database management systems (DBMS). The Database Management System (DBMS) is created by organizations in order to get the best from information systems. The ability to create, arrange, manage and manipulate data is an important aspect of any business. Coronel and Morris (2016) defined DBMS to be an assembly of process that manages a database structure and gives controlled access to the stored data. In layman's terms, a DBMS is a computer program that can store large amount of data and process them accordingly.

The Green Future – an organization focused on environmental conservation efforts, including wildlife protection, habitat restoration, and community education on sustainability practices – management system is designed using various data modelling tools. It will be stored on a database system to aid easy and better accessibility, monitoring and managing various information like registration of members, booking of projects, manage project schedules and other activities by administration for members to volunteer and support.

The system requires operations where attributes like members unique ID, name, contact information, date of birth, and project activities are stored on the systems data base to help in easy accessibility. The system will also allow members to volunteer and support projects like wildlife protection, habitat restoration, community education on sustainable practices, etc. Members are only allowed to join three to five volunteer projects per month for free depending on their membership level. There are three membership program level, thus, Basic, Silver and Gold. Members on Basic level can join three volunteer projects per month for free, Silver level members can volunteer on four projects per month for free and Gold level members can volunteer on five projects per month for free. Additional projects may require a small joining fee.

The implementation of the system, its design and development will require various tools such as the Use Case diagram, Entity Relational Diagram (ERD) as known as Entity Relational Model (ERM), Structure Query Language statements (SQL) and all relevant processes for the database for easy interaction and understanding.

Connolly and Begg (2005), stated that the database management system helps to understand and form the vital aspects of a systems architecture. They further explained that the schema is a descriptive collection of entities and attributes. These entities will use variables with one or more keys and values to effectively interact with the design and functionalities, providing information on all project activities.

1.1 Scope and Objectives of the Study

A database will be created to manage memberships and bookings for an environmental conservation organization. The system will include functionalities such as membership and staff registrations, projects booking, track member participation, manage project schedules, etc. The system will serve as a medium to enable storing, retrieving, updating and deleting of data in the organization's dataset. This system will also oversee the control of data, the database engine, and the database schema to ensure data security, integrity, concurrency, and consistent data-administration procedures. Additionally, a use case diagram, activity diagram, and an entity relational diagram – a data model – will be developed and implemented using Structure Query Language (SQL) system.

1.2 Benefits of the Case Study

This system will assist in managing complex sports activities effectively, ensuring that club members achieve utmost satisfaction. It enables administrative staff to oversee the general activities in the club like recording and updating scheduled activities while also facilitating better communication and coordination. Members will also benefit from easy access, booking, project availability, and registration. The system also helps in documentation and record keeping, ensuring smooth operation and organization within the environment conservation organization.

1.3 Design Strategy

An effective system design strategy relies on the ability to discern both the current and future system requirements. Agile methodology can be effectively applied to the development of a database management system (DBMS) assignment. Instead of trying to build the entire DBMS at once, using Agile methodology breaks the project into smaller tasks. Each task is completed in short iterations, allowing for continuous feedback and improvement.

Agile methodology encourages collaboration between developers, database administrators, and stakeholders to ensure that the Database Management System meets the requirements and expectations. This report will utilize this type of design strategy as it corresponds to the requirements of the organization. Agile methodology is flexible enough to handle changes in requirements over time and can adapt system quality, scope, and cost to meet clients demands (F. Hayat et al., 2019).

Chapter 2

2.0 System Requirement

When developing a database system, the key requirements include information about the organization, staff, and members in order to ensure good data structure. The system will enable staff to check projects booking history of members, help members keep track of their project bookings, update their profiles and check project available in the organization. The database system will be constrained by each member booking three to five projects and this is dependent on the membership level. The system will allow admin staff to record details of staff leading one or more projects, track member participation in a project, manage project schedules, and other functionalities to ensure data organization and effective management of member details and sensitive information, and member project booking.

2.1 Use-Case Diagram

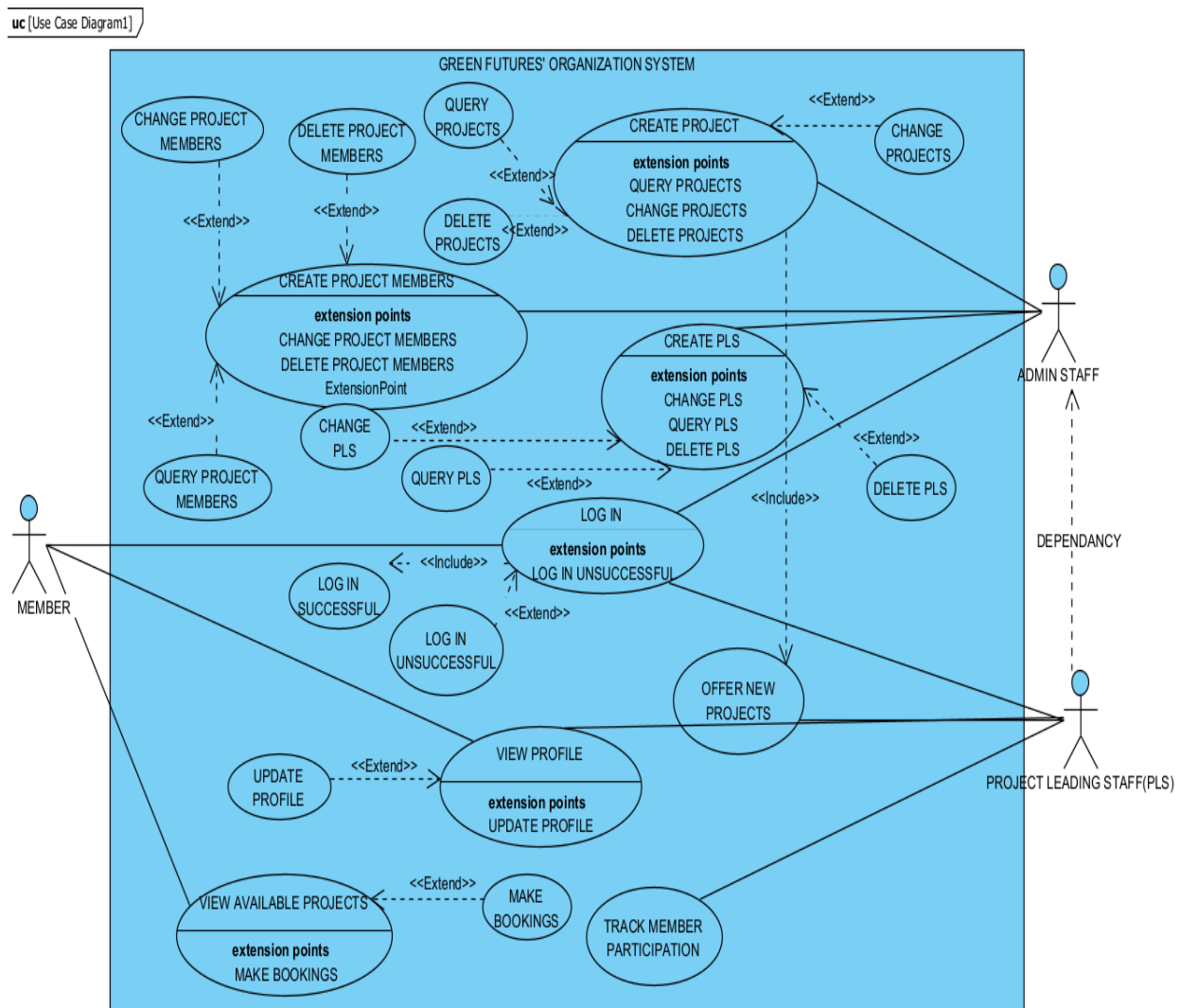


Figure 2.0: Use-Case Diagram

A use-case diagram depicts the interactions between users and the system to accomplish specific tasks or goals. It is crucial in designing and developing a software system which is a visual representation of the functional requirements of a system from the user's perspective. The use case diagram summarizes the relationships between members, admin staff and projects leading staff. A major importance of use case diagrams is their ability to aid in understanding the system's functionality, identifying user interactions, and defining system boundaries. They provide a clear overview of the system's behavior and serve as a foundation for requirements analysis and system design.

Figure 2.0 is a use case diagram of Green Futures' Environmental Conservation Organization created using Virtual Paradigm. The diagram consists of three actors, thus, members, admin staff and project leading staff (PLS). Project members can log in to view and update their profile information such as email address, medical condition, contact address, etc. Members can also view available projects and make bookings.

The administrative staff on the other hand plays a major role in the system. They have the ability to log into the system to create members, create projects, create project leading staff to spearhead a project while having access to change, query and delete these mentioned accesses.

From the system configuration, the project leading staff has a dependency on the admin staff, as the admin staff creates the staff to spearhead a project and also creates projects for a staff to lead. The project leading staff can log in to track member participation, offer new projects, and update their profile.

2.2 Activity Diagram

An activity diagram is a visual representation used to illustrate the flow of control within a system, showing the sequence involved in the execution of activities. Activity diagrams are widely used in software engineering for modelling business processes, system workflows, and behavior scenarios. An important aspect of activity diagrams is their ability to model the flow of activities within a system, illustrating how distinct components interact and behave over time. They are useful for understanding complex processes, identifying bottlenecks, and optimizing system performance.

In general, use case diagrams and activity diagrams are valuable tools in software development, providing visual representation of system functionalities and behavior. These diagrams help in requirements analysis, system design, and communication among stakeholders, contributing to the overall success of software system projects.

In designing the activity diagram for Green Futures' Organization using Visual Paradigm, the following procedures were employed to provide insights into the rationale and relationships of the system at each stage. The flow of the activity diagram below for making bookings is as follows:

- A member of Green Futures' organization will have to enter his or her credentials to login to the system.
- The system will prompt the user to enter login credentials to validate their access. A decision node is used after the user enters login credentials to determine if login details are

correct. If credentials are wrong, the system will prompt the user to enter the credentials again.

- After a successful login to the system, the user (member) can view their dashboard. The user can then view projects and book projects. If a project is not available, the user will be asked to view other projects. However, if the project is available, the user will be prompted to book a project which can be free or will come at a cost depending on their membership level and the number of projects the user has booked per month.
- The system will check user's membership level and if a user has reached their free projects booking per month, users will be asked to make payment to be able to book that project. However, if a user has not reached their free project booking limit, they will be asked to confirm project booking and will receive a display message about booking after which the user will finally end the flow.
- If the user has reached his or her limit, a payment prompt will pop up. If the user decides not to pay, booking will be decline and a resolve issue prompt will be displayed for the user to make payments or end flow.
- If the user makes payment, the user is then prompted to confirm booking with a merging node merging the flow and a booking message will be displayed to end the flow.

Below is a visual representation of the described activity diagram for booking a project.

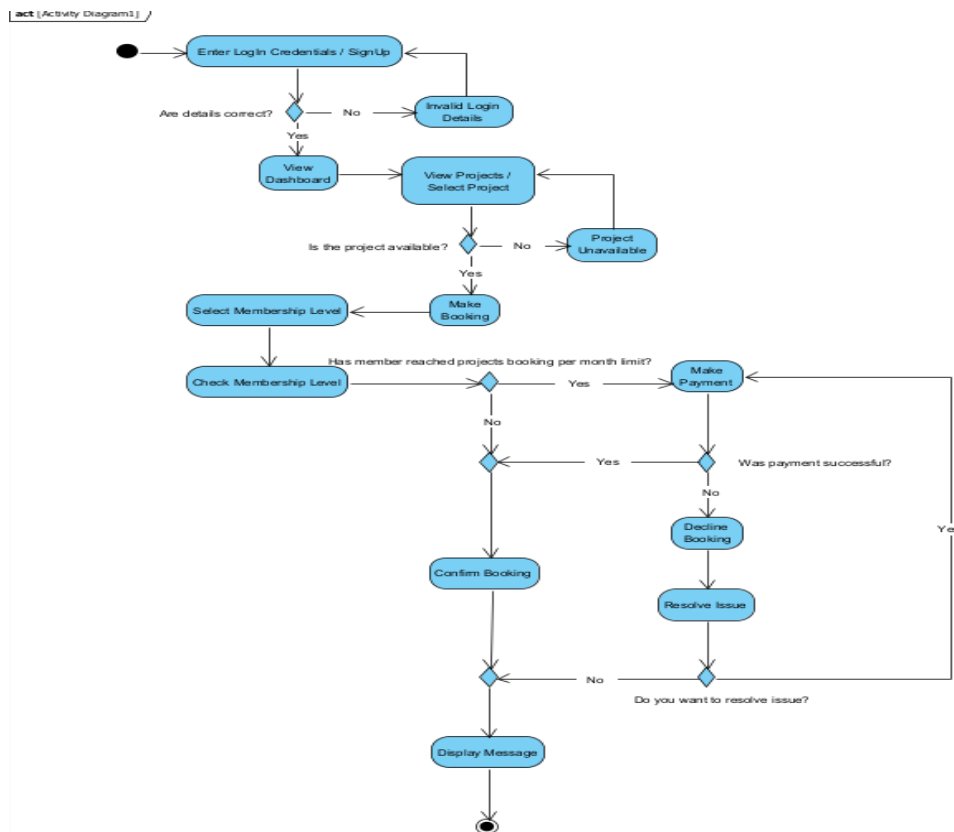


Figure 2.1: Activity Diagram

Chapter 3

3.0 System Design

The process of defining the architecture, components, modules, interfaces, and data for a system to satisfy specified requirements can be termed as System Design. This involves translating user requirements into a detailed blueprint that guides the implementation phase. System Design is not only a vital step in the development of the system but also provides the backbone to handle exceptional scenarios because it represents the business logic of software. Successful development of a functional system is reliant upon a carefully considered design process as systems are easier to create and use (Coronel et al., 2016). In order to achieve this goal, an Entity Relationship Model (ERM) will be used and represented using Entity Relationship Diagram (ERD). Entity Relationship Diagrams (ERD) are used to represent the details about entities, attributes, and relationships between applications. Primary keys and foreign keys are heavily used in these types of systems with the former used to uniquely identify records and the latter used to connect the tables and enable table joins.

3.1 Data Modelling

Data Modelling helps in the visual representation of data and the relationship between entities. This representation includes the entity relation model (ERM) that depicts how entities interact with each other within a system. It is a primary model in designing RDBMS. These entities, including Membership, Member, Staff, Project, Project Booking, and Payment, play a pivotal role in the organization's operations and interactions. Members engage in booking activities while Staff oversees various aspects especially admin staff roles and project leading staff. To handle this, an attribute named *staff_role* is introduced in the ERD to differentiate staff roles.

The entities and corresponding attributes involved in the Green Futures' system management are:

- Membership: The attributes for the entity are Membership_ID, Membership_Type, Membership_Fee, Membership_Start date, Membership_End date, and Member_ID.
- Member: The attributes for the entity member are Member_ID, FirstName, Surname, Date of Birth (DoB), Home Address, Email, Telephone, Gender, Nationality, MedicalCondition.
- Staff: The attributes for this entity are Staff_ID, FirstName, Surname, Email, Telephone, Home Address, Gender, Nationality, StaffRole.
- Project: Project_Code, Project_Title, Project_Capacity, Project_Start date, Project_End date, Project_Type and Staff_ID are the attributes for this entity.
- Project_Booking: This entity has the following attributes, Project_BookingID, BookingDate, BookingTime, Staff_ID, Member_ID, Project_Code.
- Payment: The attributes for payment are Payment_ID, Payment_Date, Payment_Amount, Payment_Method, Member_ID.

3.2 Entity Relationship Diagram

Figure 3.0 below is a structural representation of the entity relational diagram using virtual paradigm. It gives a visual overview of the following relationship between entities.

Member:

- Has one-to-many relationship with membership. Each member can have multiple memberships, but each membership belongs to only one member.
- Has a many-to-many relationship with staff. A member can be taught by many project leading staff and many projecting leading staff can teach the same member.
- Has many-to-many relationship with project. A member can book many projects and one project can have many members.
- Has a one-to-many relationship with payment. Each member can have multiple project payments, but each payment made belongs to only one member.

Staff:

- Has many-to-many relationship with project. A staff can lead many projects and a project can have many project leading staff.
- Has many-to-many relationship with member. A staff can lead many members and a member can be led by many staff.

Project:

- Has many-to-many relationship with a staff. A project can be led by many staff and a staff can lead many projects.
- Has a many-to-many relationship with member. A project can have many members and a member can book many projects.

Membership:

- Has a many-to-one relationship with member. A member can have multiple membership.

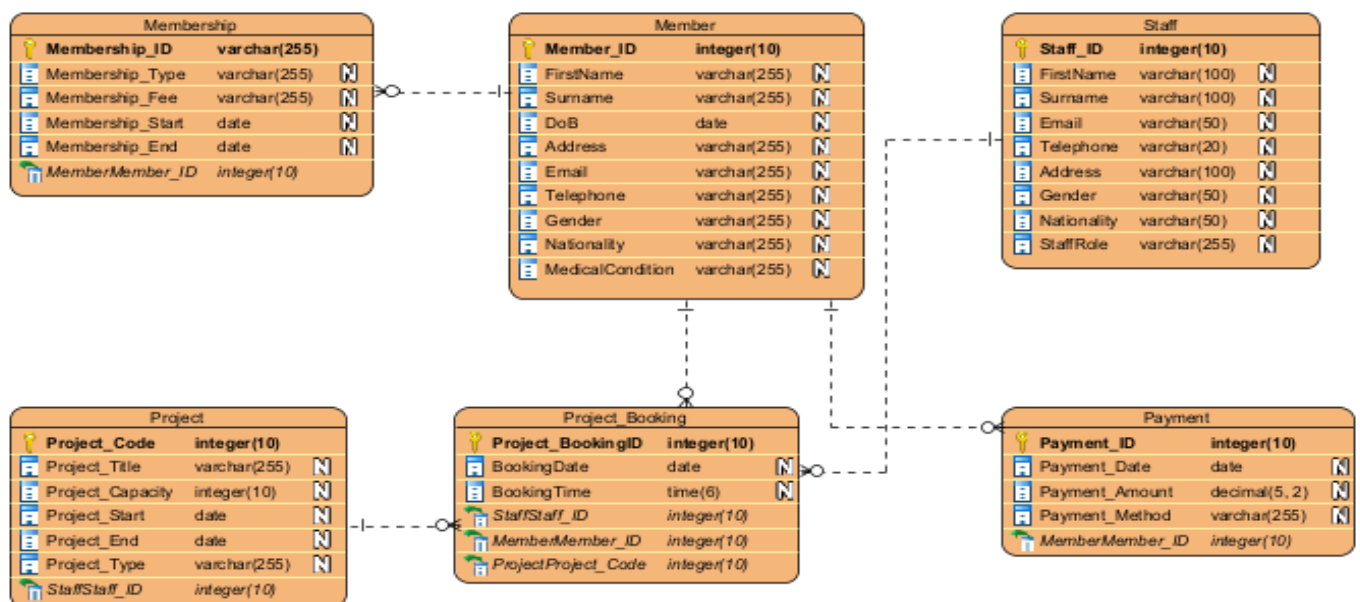


Figure 3.0: Entity Relationship Diagram

Chapter 4

4.0 System Implementation

For implementing the system, a Relational Database Management System called MySQL which uses Structure Query Language (SQL) queries to access data in databases is used, while a web application called phpMyAdmin is used to manage MySQL database.

The following section describes the SQL statements that were used to create database and the table to insert the data into. Below is the pictorial view of how the entities in the database was implemented.

4.1 Creating Database for Green Futures'

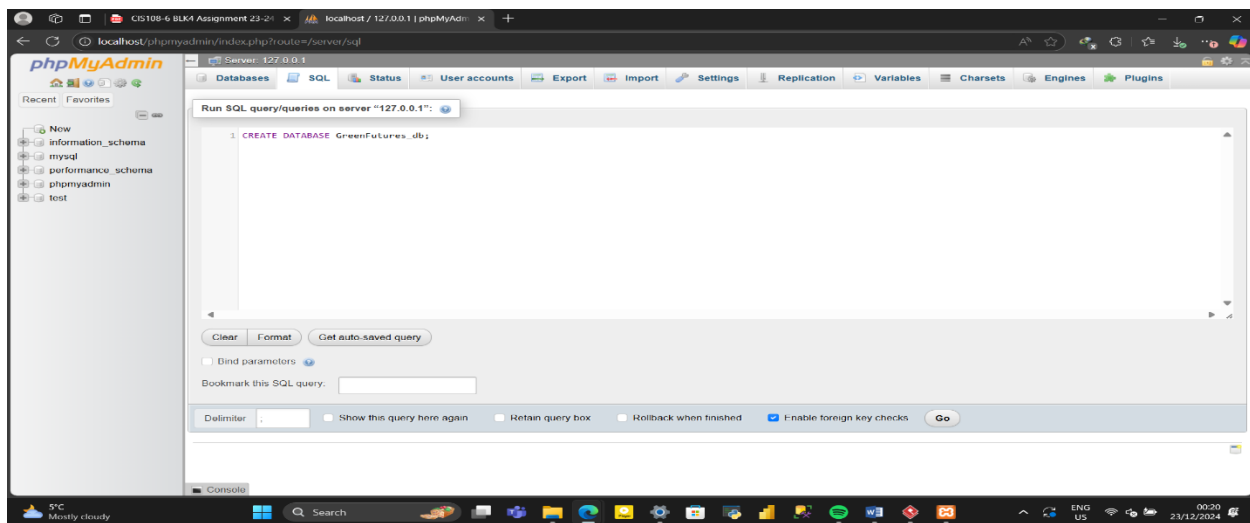


Figure 4.1.0: SQL Statement to Create Green Futures' Database

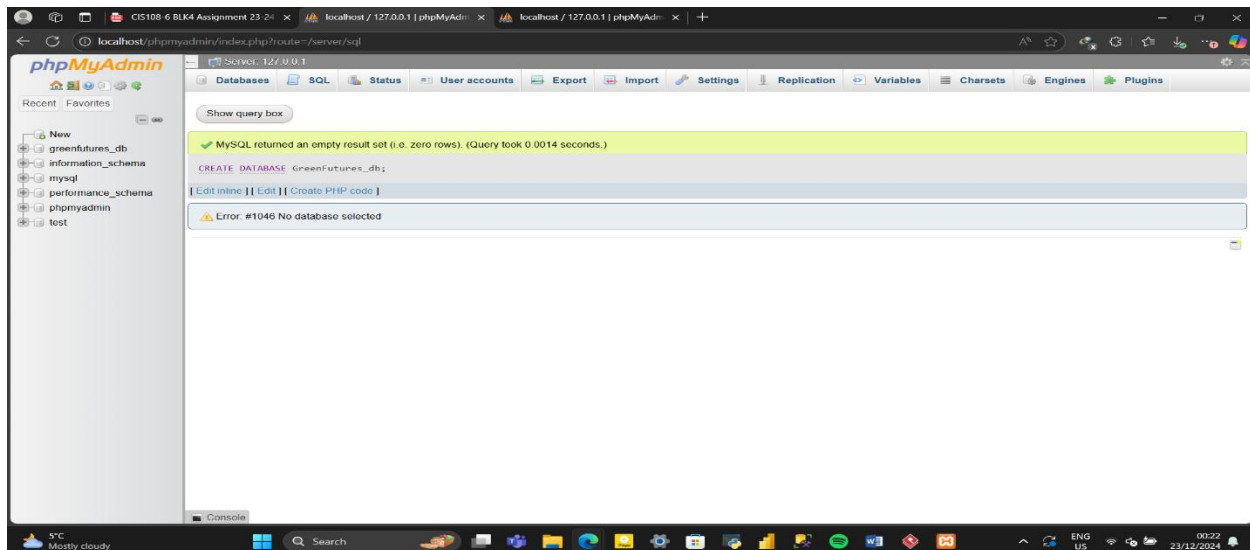


Figure 4.1.1: Successful Creation of Green Futures' Database

4.2 Membership Table

```
CREATE TABLE Membership (  
    Membership_ID VARCHAR(255) PRIMARY KEY,  
    Membership_Type VARCHAR(255) NOT NULL,  
    Membership_Fee VARCHAR(255) NOT NULL,  
    Membership_Start DATE,  
    Membership_End DATE,  
    Member_ID INT(10),  
    FOREIGN KEY (Member_ID) REFERENCES Member (Member_ID) ON DELETE  
    CASCADE  
)ENGINE=INNODB;
```

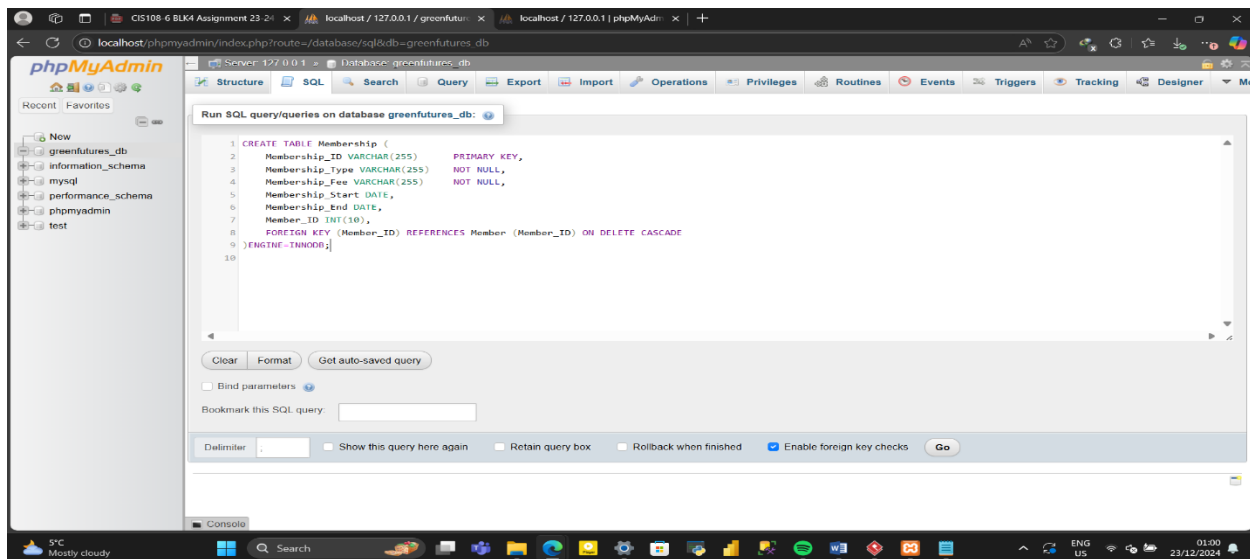


Figure 4.2.0: SQL Statement Code to Create Membership Table

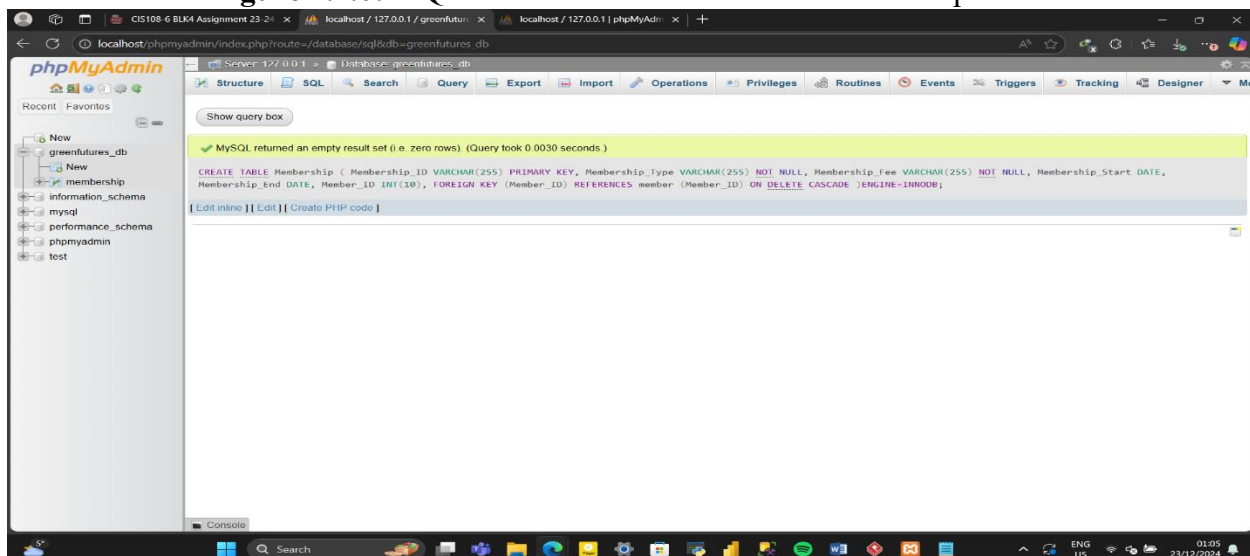


Figure 4.2.1: Successful Creation of Membership Table

4.3 Member Table

```
CREATE TABLE Member (  
    Member_ID INT(10) PRIMARY KEY,  
    FirstName VARCHAR(255) NOT NULL,  
    Surname VARCHAR(255) NOT NULL,  
    DoB DATE,  
    Address VARCHAR(255) NOT NULL,  
    Email VARCHAR(255) NOT NULL,  
    Telephone VARCHAR(255) NOT NULL,  
    Gender VARCHAR(255) NOT NULL,  
    Nationality VARCHAR(255) NOT NULL,  
    MedicalCondition VARCHAR(255) NOT NULL  
)ENGINE=INNODB;
```

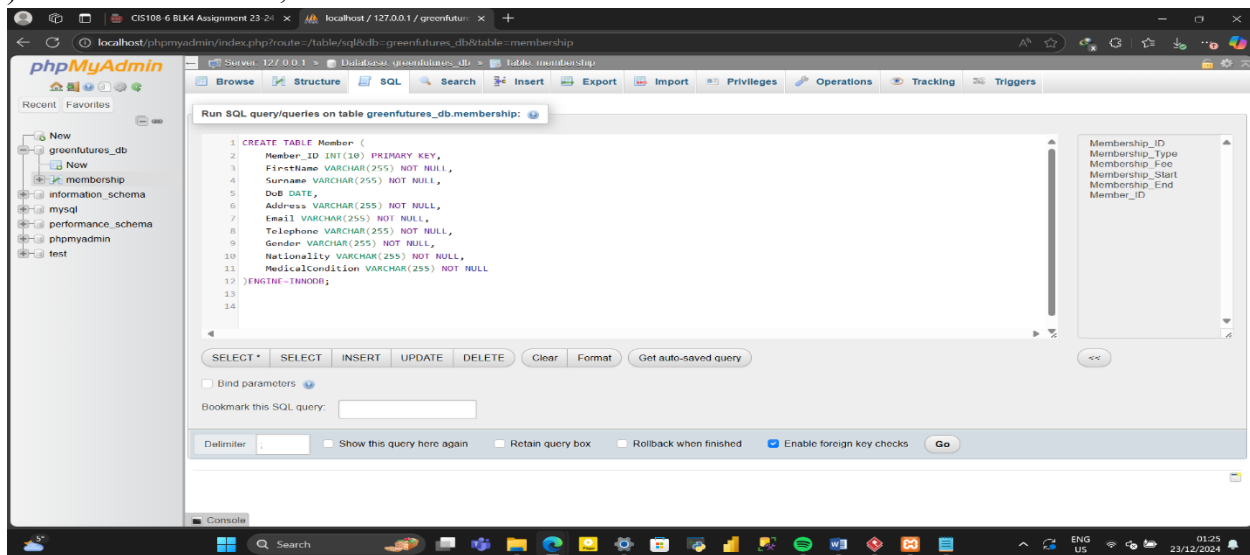


Figure 4.3.0: SQL Statement Code to Create Member Table

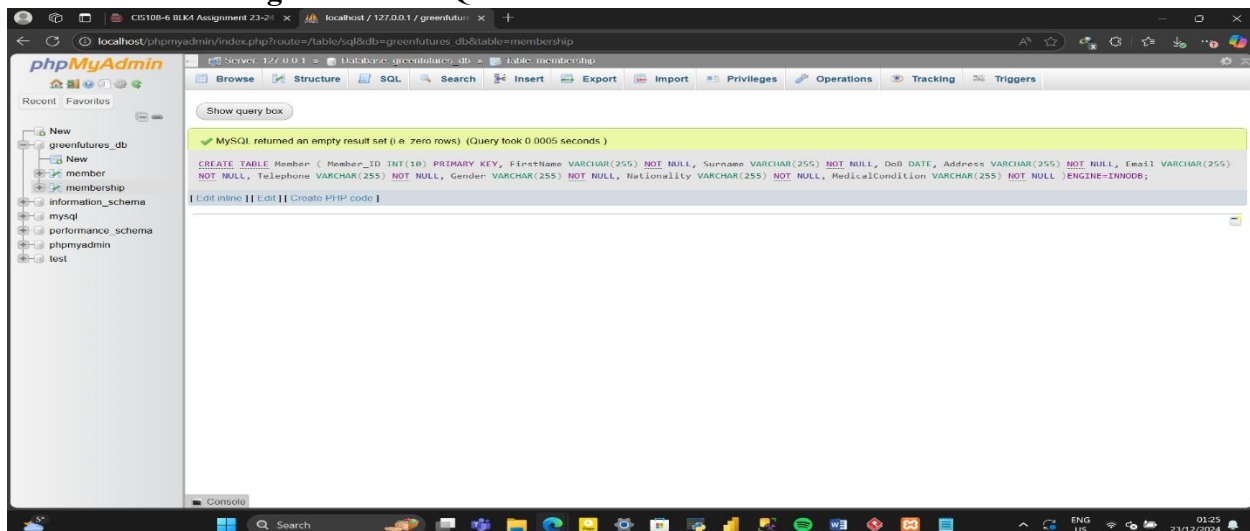


Figure 4.3.1: Successful Creation of Member Table

4.4 Staff Table

```
CREATE TABLE Staff (  
    Staff_ID INT(10) PRIMARY KEY,  
    FirstName VARCHAR(100) NOT NULL,  
    Surname VARCHAR(100) NOT NULL,  
    Email VARCHAR(50) NOT NULL,  
    Telephone VARCHAR(20) NOT NULL,  
    Address VARCHAR(100) NOT NULL,  
    Gender VARCHAR(50) NOT NULL,  
    Nationality VARCHAR(50) NOT NULL,  
    StaffRole VARCHAR(255) NOT NULL  
)ENGINE=INNODB;
```

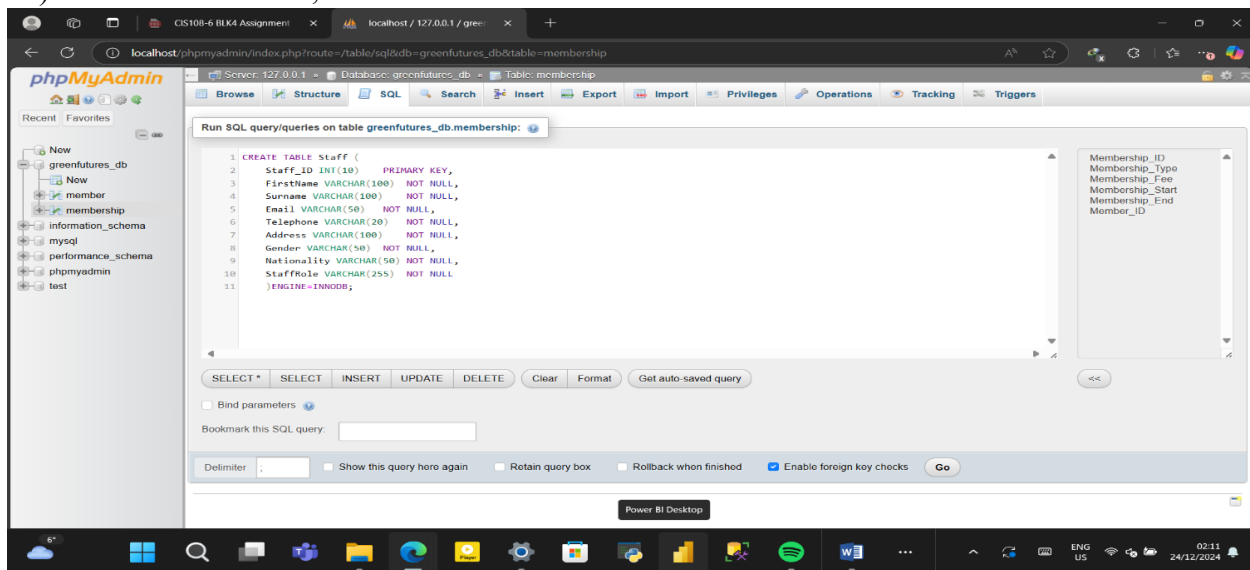


Figure 4.4.0: SQL Statement Code to Create Member Table

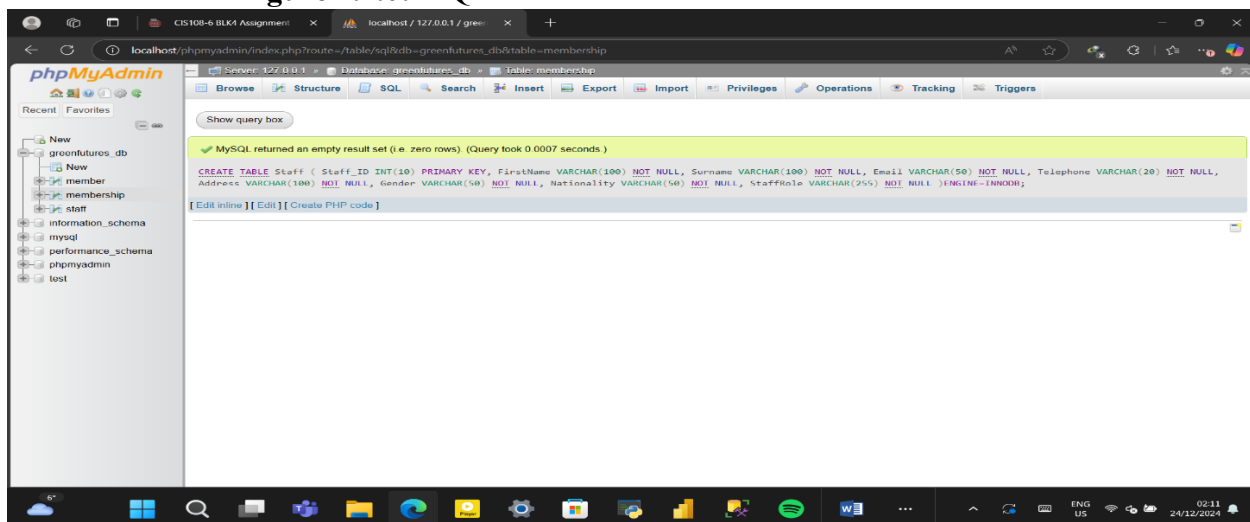


Figure 4.4.1: Successful Creation of Staff Table

4.5 Project Table

```
CREATE TABLE Project (  
    Project_Code INT(10) PRIMARY KEY,  
    Project_Title VARCHAR(255) NOT NULL,  
    Project_Capacity INT(10) NOT NULL,  
    Project_Start DATE,  
    Project_End DATE,  
    Project_Type VARCHAR(255) NOT NULL,  
    Staff_ID INT(10),  
    FOREIGN KEY (Staff_ID) REFERENCES Staff (Staff_ID) ON DELETE CASCADE  
)ENGINE=INNODB;
```

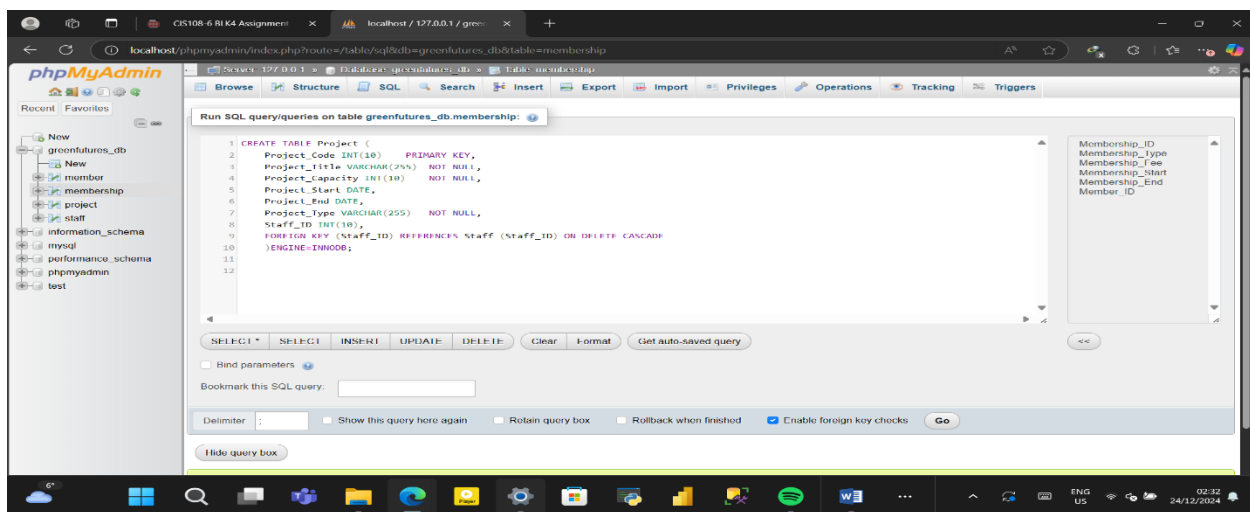


Figure 4.5.0: SQL Statement Code to Create Project Table

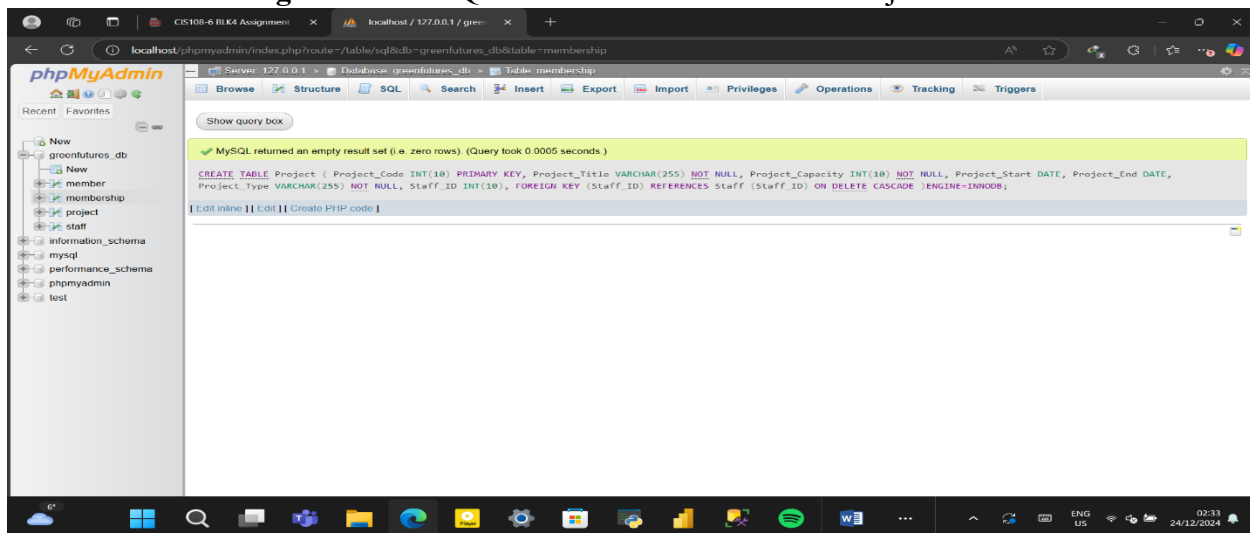


Figure 4.5.1: Successful Creation of Project Table

4.6 Project_Boooking Table

```
CREATE TABLE Project_Booking (  
    Project_BookingID INT(10) PRIMARY KEY,  
    BookingDate DATE,  
    BookingTime TIME(6),  
    Staff_ID INT(10),  
    Member_ID INT(10),  
    Project_Code INT(10),  
    FOREIGN KEY (Member_ID) REFERENCES Member (Member_ID) ON DELETE  
    CASCADE,  
    FOREIGN KEY (Staff_ID) REFERENCES Staff (Staff_ID) ON DELETE CASCADE,  
    FOREIGN KEY (Project_Code) REFERENCES Project (Project_Code) ON DELETE  
    CASCADE  
    )ENGINE=INNODB;
```

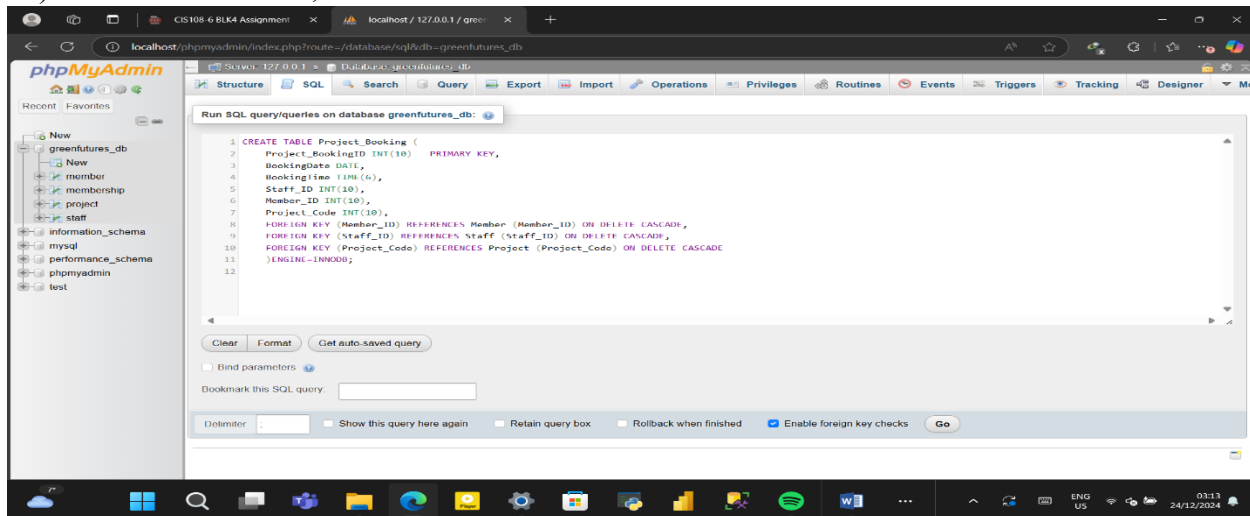


Figure 4.6.0: SQL Statement Code to Create Project Booking Table

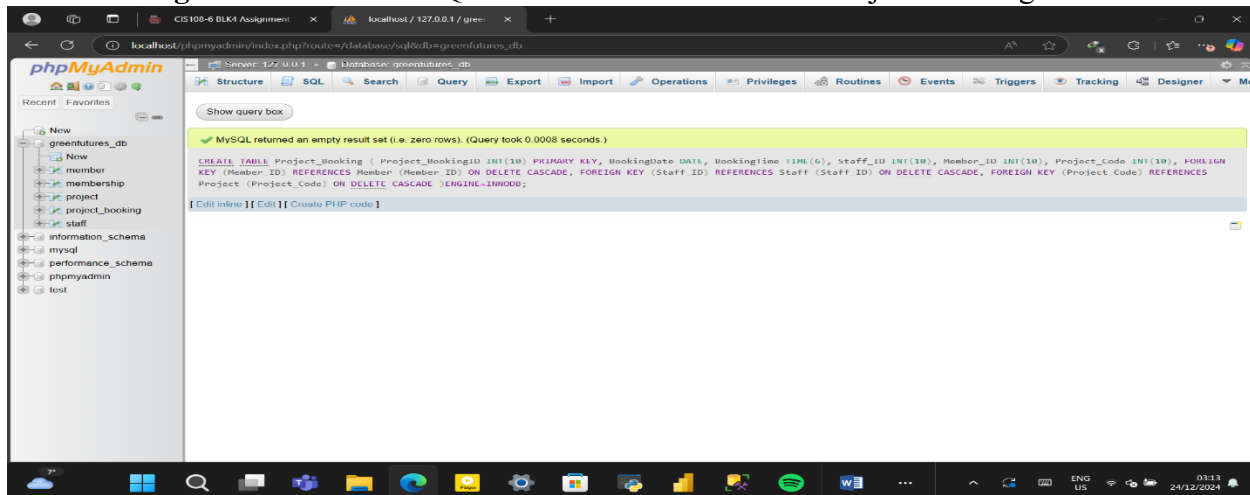


Figure 4.6.1: Successful Creation of Project Booking Table

4.7 Payment Table

```
CREATE TABLE Payment (  
    Payment_ID INT(10) PRIMARY KEY,  
    Payment_Date DATE,  
    Payment_Amount DECIMAL(5,2),  
    Payment_Method VARCHAR(255) NOT NULL,  
    Member_ID INT(10),  
    FOREIGN KEY (Member_ID) REFERENCES Member (Member_ID) ON DELETE  
    CASCADE  
)ENGINE=INNODB;
```

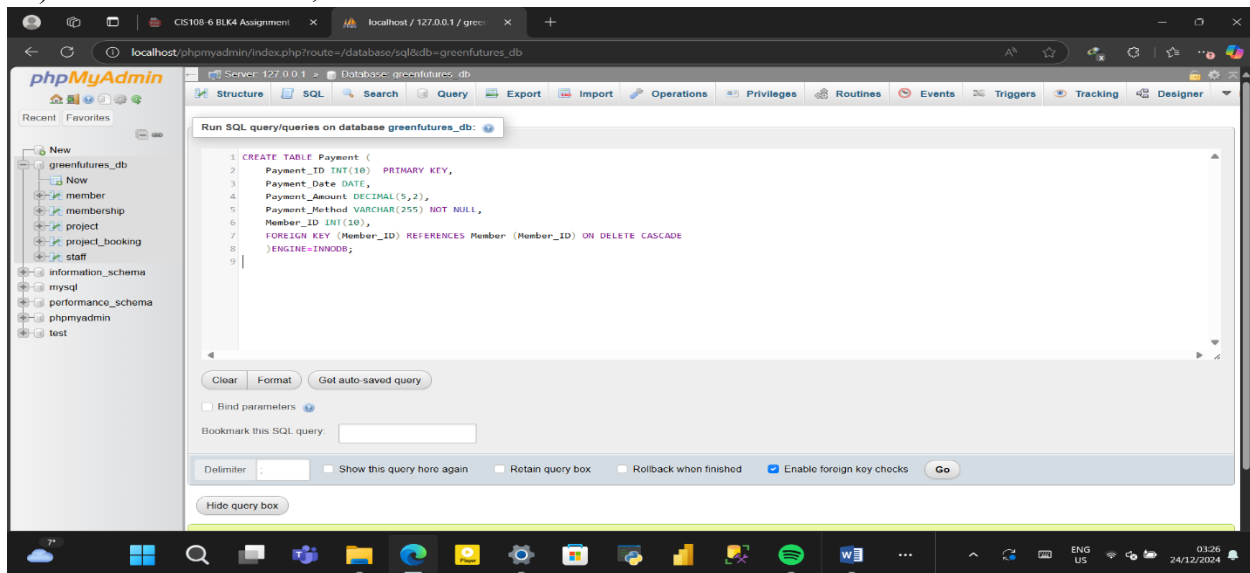


Figure 4.7.0: SQL Statement Code to Create Payment Table

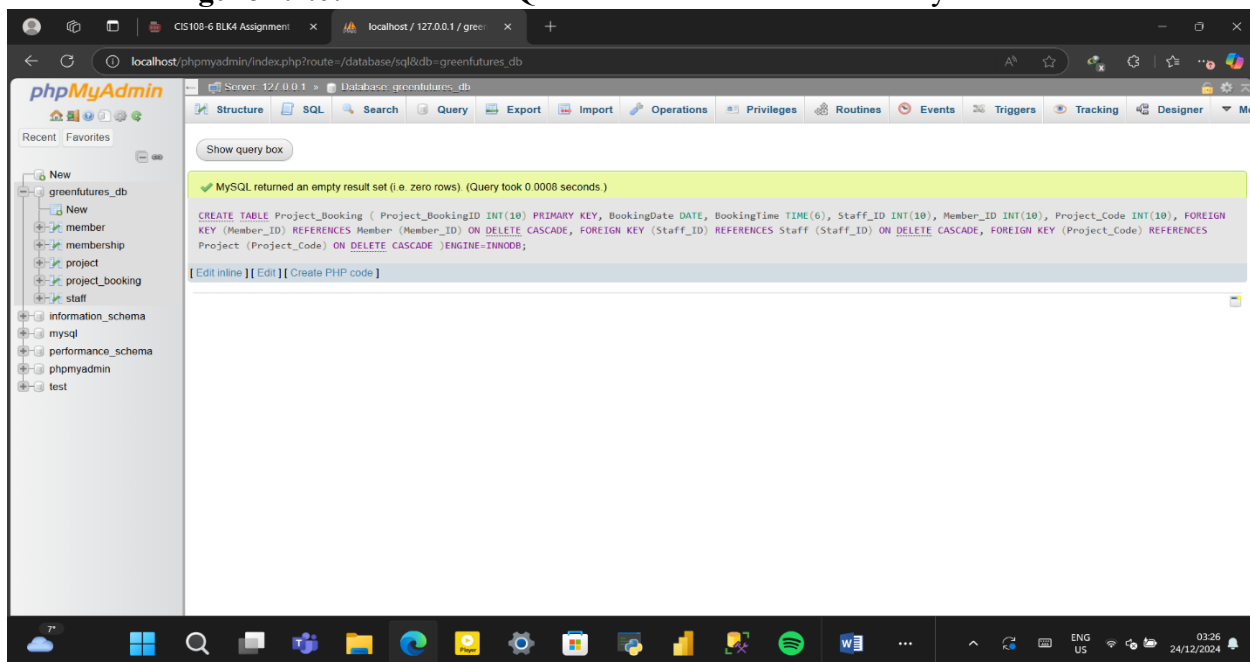


Figure 4.7.1: Successful Creation of Payment Table

4.8 Tables Relation

The figure 4.8.1 below shows the relationship between the tables successfully created in the database.

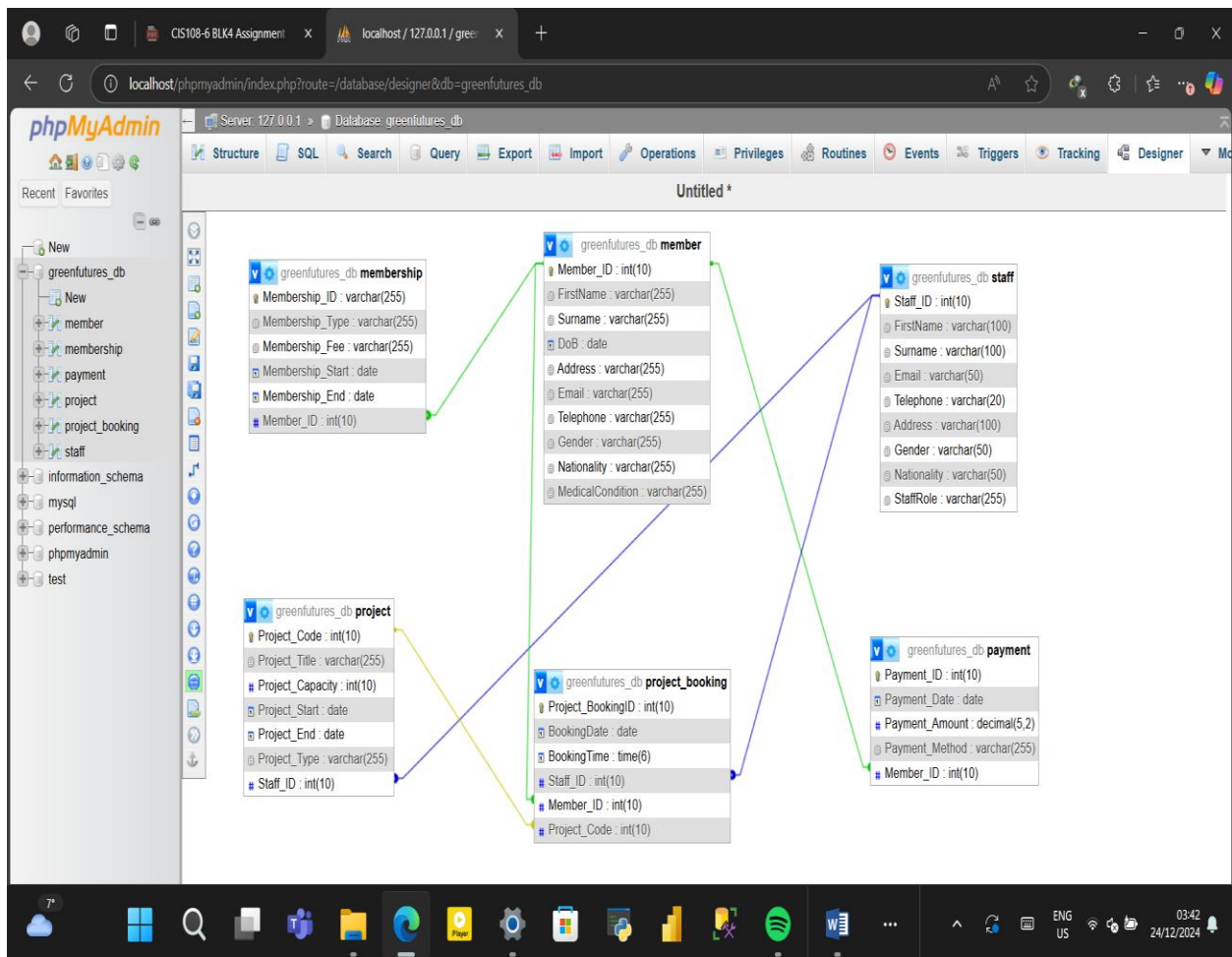


Figure 4.8.1: Tables Relation

4.9 Data Insertion

Similar to creating tables using SQL statements, we use INSERT TO 'The Table'() followed by VALUES(); which are key SQL statements used suppose I want to insert data into a table.

The figures below show a visual representation of how to input data into the database.

4.9.1 Member Table Data Insertion

INSERT INTO Member

(Member_ID, FirstName, Surname, DoB, Address, Email, Telephone, Gender, Nationality, MedicalCondition)

VALUES

(1001, 'Imoh', 'Umoro', '1994-04-25', '72 Crosslands Stantonbury Road MK14 7AX', 'imoh.umoro@gmail.com', 7324538856, 'M', 'Nigeria', 'Diabetes'),

(1002, 'Aisha', 'Abdul', '1990-09-12', '45 Greenfield Road, Birmingham, B18 6NF', 'aisha.abdul@example.com', 7894561230, 'F', 'Nigeria', 'Hypertension'),

(1003, 'James', 'Owusu', '1988-03-07', '16 Kingsway, London, SW1E 6DD', 'james.owusu@example.com', 7723456789, 'M', 'Ghana', 'Asthma'),

(1004, 'Sophia', 'Musa', '1995-11-22', '89 Bridge Street, Manchester, M4 5DL', 'sophia.musa@example.com', 7312345678, 'F', 'Nigeria', 'None'),

(1005, 'Kwame', 'Asare', '1985-01-15', '32 Broad Street, Accra, Ghana', 'kwame.asare@example.com', 233545678901, 'M', 'Ghana', 'Sickle Cell Anemia'),

(1006, 'Emily', 'Smith', '1992-06-10', '24 Baker Street, Nottingham, NG1 6BQ', 'emily.smith@example.com', 7789123456, 'F', 'United Kingdom', 'None'),

(1007, 'Victor', 'Okonkwo', '1989-08-30', '50 Oakwood Avenue, Leeds, LS6 1BA', 'victor.okonkwo@example.com', 7896543210, 'M', 'Nigeria', 'Epilepsy'),

(1008, 'Amma', 'Nyarko', '1993-04-19', '12 Ridge Street, Kumasi, Ghana', 'amma.nyarko@example.com', 233244567890, 'F', 'Ghana', 'Allergy'),

(1009, 'Daniel', 'Brown', '1987-12-05', '78 High Street, Cardiff, CF10 1PX', 'daniel.brown@example.com', 7890123456, 'M', 'United Kingdom', 'None'),

(1010, 'Fatima', 'Yusuf', '1996-02-11', '60 Victoria Road, Abuja, Nigeria', 'fatima.yusuf@example.com', 234706543210, 'F', 'Nigeria', 'Hypertension');

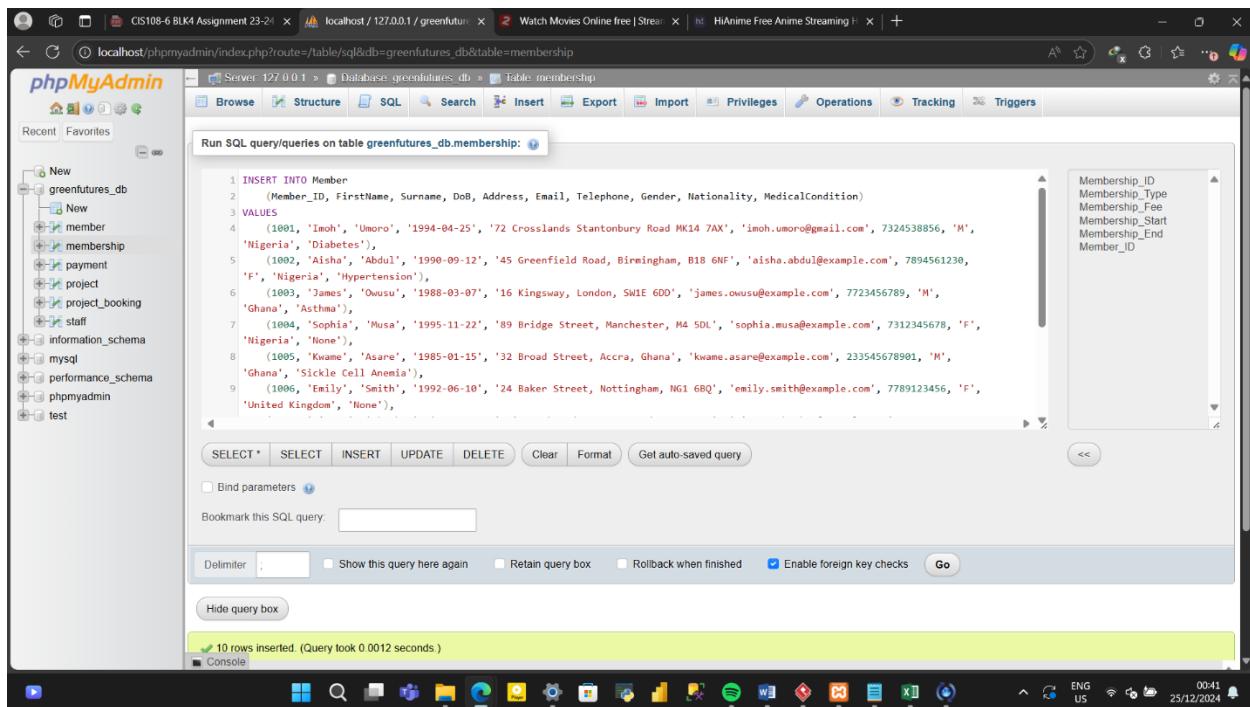


Figure 4.9.1.1: Data Entry for Member Table

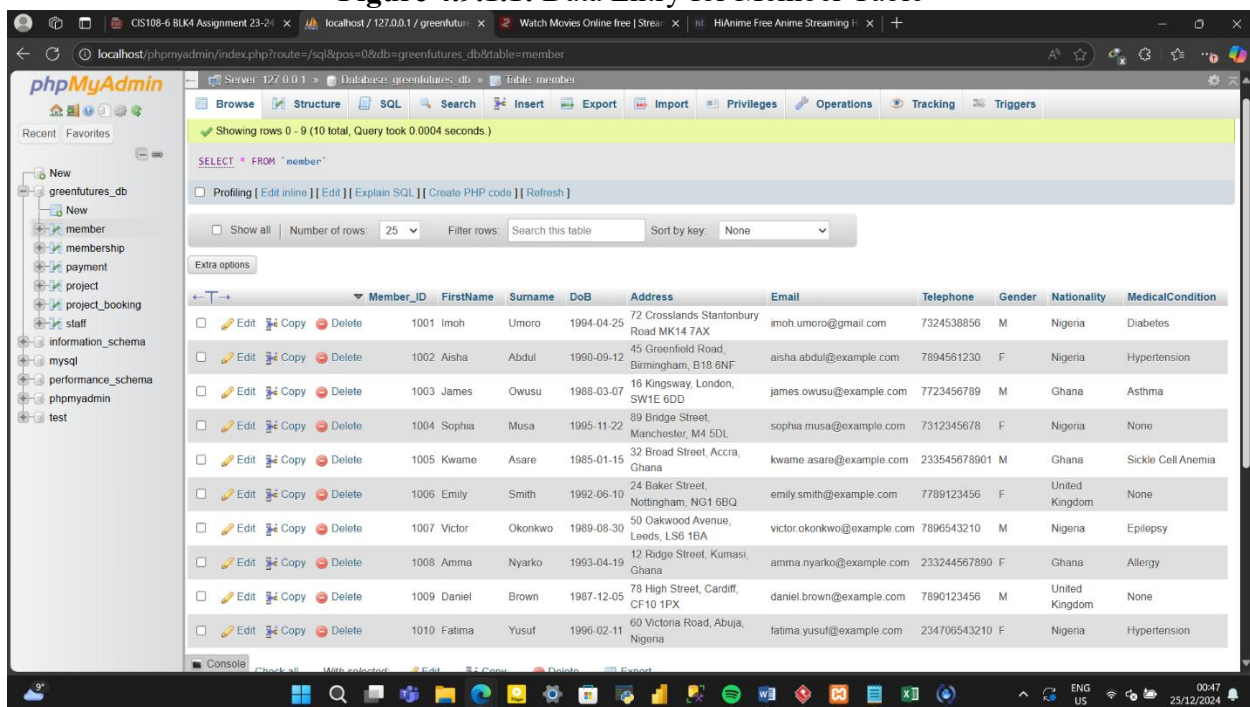


Figure 4.9.1.2: Data Successfully Entered into Member Table

4.9.2 Membership Table Data Insertion

INSERT INTO Membership

(Membership_ID, Membership_Type, Membership_Fee, Membership_Start, Membership_End, Member_ID)

VALUES

(2001, 'Silver', '70', '2024-06-27', '2025-06-27', 1002),
(2002, 'Silver', '70', '2023-03-01', '2024-03-01', 1003),
(2003, 'Basic', '40', '2023-07-20', '2024-07-20', 1004),
(2004, 'Silver', '70', '2023-11-05', '2024-11-05', 1005),
(2005, 'Silver', '70', '2022-12-12', '2023-12-12', 1006),
(2006, 'Gold', '100', '2023-06-10', '2024-06-10', 1007),
(2007, 'Basic', '40', '2023-09-01', '2024-09-01', 1009),
(2008, 'Basic', '40', '2024-02-15', '2025-02-15', 1008),
(2009, 'Basic', '40', '2023-05-20', '2024-05-20', 1009),
(2010, 'Gold', '100', '2023-10-25', '2024-10-25', 1001);

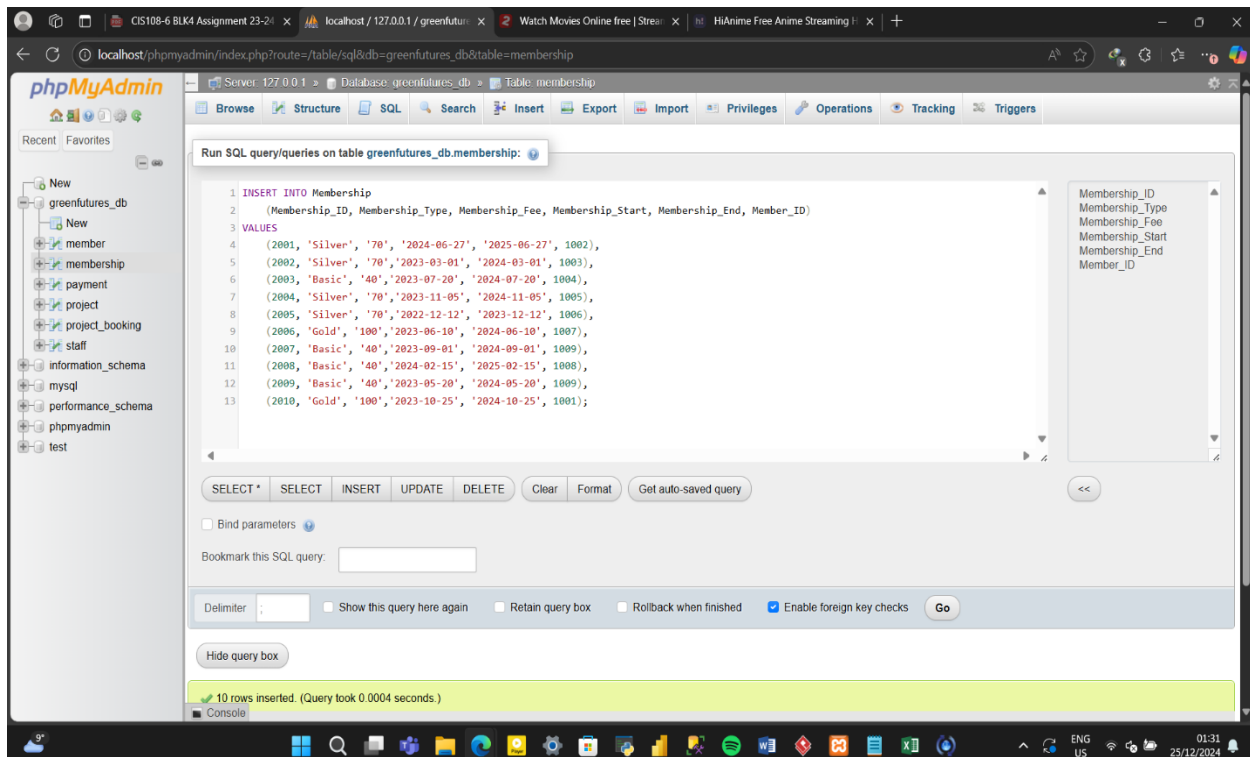


Figure 4.9.2.1: Data Entry for Membership Table

Showing rows 0 - 9 (10 total, Query took 0.0004 seconds)

```
SELECT * FROM `membership`
```

Extra options

	Membership_ID	Membership_Type	Membership_Fee	Membership_Start	Membership_End	Member_ID
☐	2001	Silver	70	2024-06-27	2025-06-27	1002
☐	2002	Silver	70	2023-03-01	2024-03-01	1003
☐	2003	Basic	40	2023-07-20	2024-07-20	1004
☐	2004	Silver	70	2023-11-05	2024-11-05	1005
☐	2005	Silver	70	2022-12-12	2023-12-12	1006
☐	2006	Gold	100	2023-06-10	2024-06-10	1007
☐	2007	Basic	40	2023-09-01	2024-09-01	1009
☐	2008	Basic	40	2024-02-15	2025-02-15	1008
☐	2009	Basic	40	2023-05-20	2024-05-20	1009
☐	2010	Gold	100	2023-10-25	2024-10-25	1001

Query results operations

Figure 4.9.2.2: Data Successfully Entered into Membership Table

4.9.3 Staff Table Data Insertion

INSERT INTO Staff

(Staff_ID, FirstName, Surname, Email, Telephone, Address, Gender, Nationality, StaffRole)
VALUES

(3001, 'Emily', 'Jackson', 'emi.jackson54@gmail.com', 7443247754, '101 Dorrington Close, Luton, LU3 1XP', 'F', 'Canada', 'Receptionist')
 (3002, 'Kwesi', 'Boateng', 'kwesi.boateng21@gmail.com', 233246789012, '22 Ridge Street, Kumasi, Ghana', 'M', 'Ghana', 'Administrator')
 (3003, 'Daniel', 'Nguyen', 'daniel.nguyen88@gmail.com', 7123456789, '12 Victoria Road, Cardiff, CF10 2HQ', 'M', 'Vietnam', 'IT Support')
 (3004, 'Sophia', 'White', 'sophia.white77@gmail.com', 7789123456, '78 King Street, Manchester, M1 2AP', 'F', 'United Kingdom', 'Project Leader')
 (3005, 'Fatima', 'Ahmed', 'fatima.ahmed90@gmail.com', 7894561230, '15 Oakwood Avenue, Birmingham, B15 2TT', 'F', 'Nigeria', 'Project Leader')
 (3006, 'Amara', 'Okafor', 'amara.okafor22@gmail.com', 234701234567, '34 Queen Street, Abuja, Nigeria', 'F', 'Nigeria', 'Project Leader')
 (3007, 'Evelyn', 'Mensah', 'evelyn.mensah15@gmail.com', 233547890123, '40 Chapel Hill, Cape Coast, Ghana', 'F', 'Ghana', 'Project Leader')
 (3008, 'James', 'Brown', 'james.brown44@gmail.com', 7890123456, '101 High Street, Edinburgh, EH1 2AB', 'M', 'United Kingdom', 'Project Leader')

(3009, 'Layla', 'Khan', 'layla.khan99@gmail.com', 7412589630, '85 Greenfield Road, Glasgow, G1 1AP', 'F', 'Pakistan', 'Project Leader')

(3010, 'Thomas', 'Owusu', 'thomas.owusu12@gmail.com', 233245678901, '18 Maple Street, Accra, Ghana', 'M', 'Ghana', 'Project Leader')

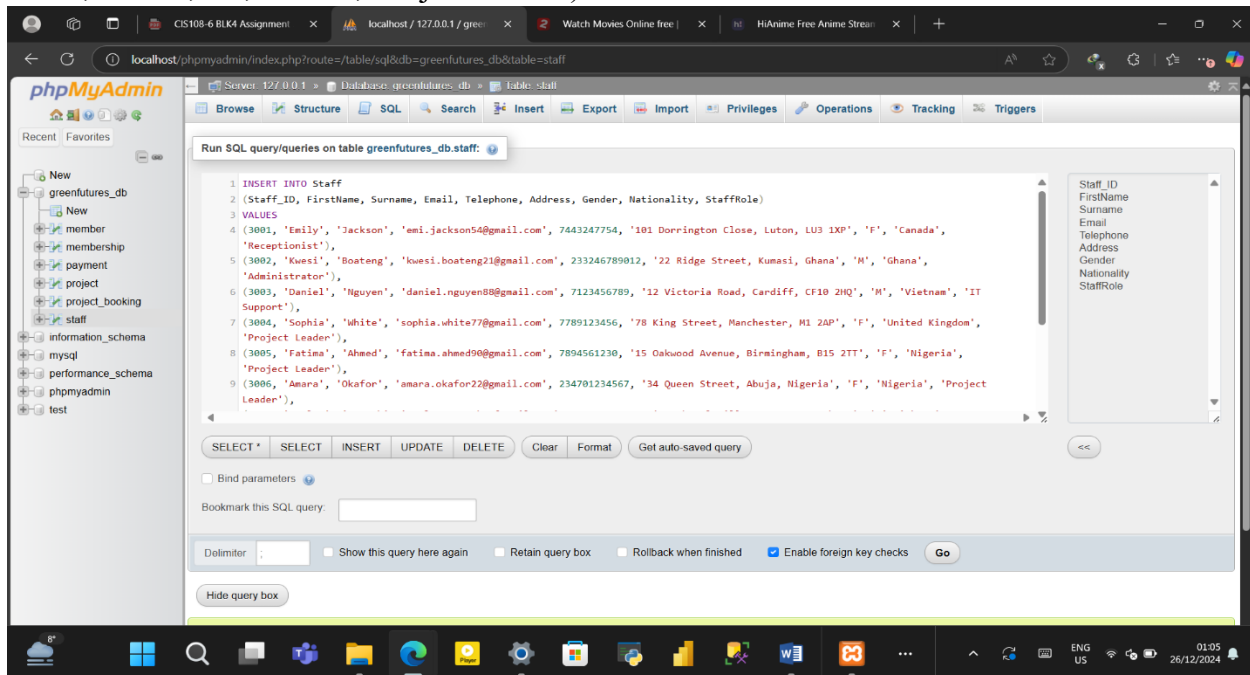


Figure 4.9.3.1: Data Entry for Staff Table

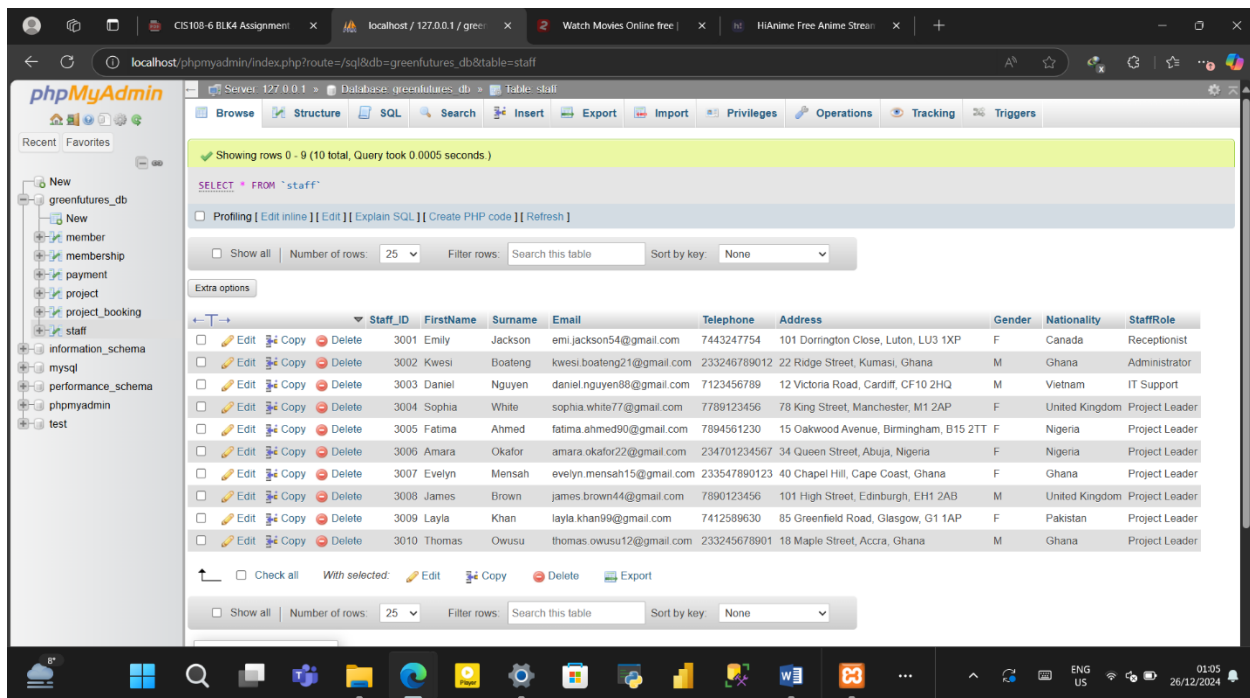


Figure 4.9.3.2: Data Successfully Entered into Staff Table

4.9.4 Project Table Data Insertion

INSERT INTO Project

(Project_Code, Project_Title, Project_Capacity, Project_Start, Project_End, Project_Type, Staff_ID)

VALUES

(4001, 'Plastic Waste Management', 50, '2025-02-20', '2025-04-19', 'Pollution Control and Waste Management', 3005),

(4002, 'Forest Restoration Initiative', 100, '2025-01-15', '2025-03-15', 'Forest Conservation', 3004),

(4003, 'Urban Green Spaces Expansion', 150, '2025-06-01', '2025-08-31', 'Urban Environmental Management', 3004),

(4004, 'Wildlife Protection Campaign', 80, '2025-02-01', '2025-04-31', 'Biodiversity Conservation', 3008),

(4005, 'Wetlands Recovery Program', 60, '2025-04-15', '2025-06-14', 'Ecosystem Restoration', 3006),

(4006, 'Solar Energy Initiative', 20, '2025-07-20', '2025-09-19', 'Renewable Energy and Climate Change Mitigation', 3006),

(4007, 'Plastic Free Oceans Drive', 120, '2025-05-10', '2025-07-09', 'Pollution Control and Waste Management', 3010),

(4008, 'Community Forestry Program', 90, '2025-03-01', '2025-05-28', 'Community and Ecosystem-Based Conservation', 3009),

(4009, 'Energy Efficiency Awareness', 35, '2025-08-25', '2025-10-24', 'Energy Conservation', 3008),

(4010, 'Sustainable Farming Initiative', 100, '2025-10-15', '2025-12-14', 'Agricultural Sustainability', 3007);

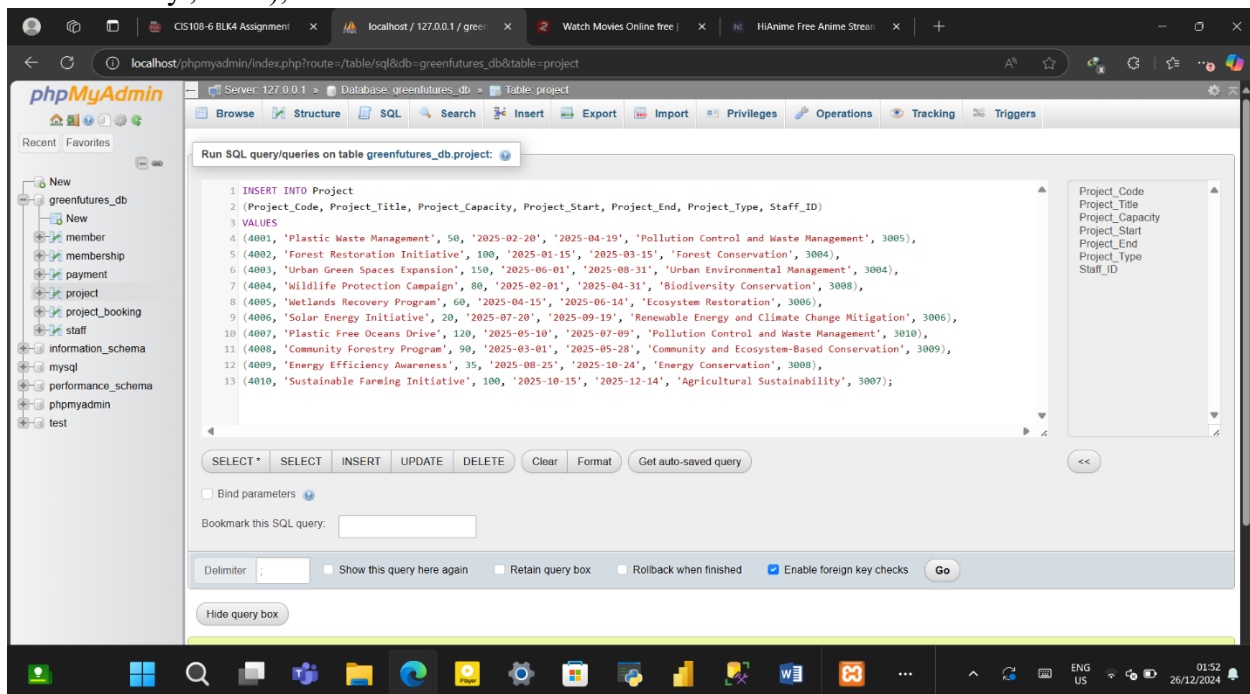


Figure 4.9.4.1: Data Entry for Project Table

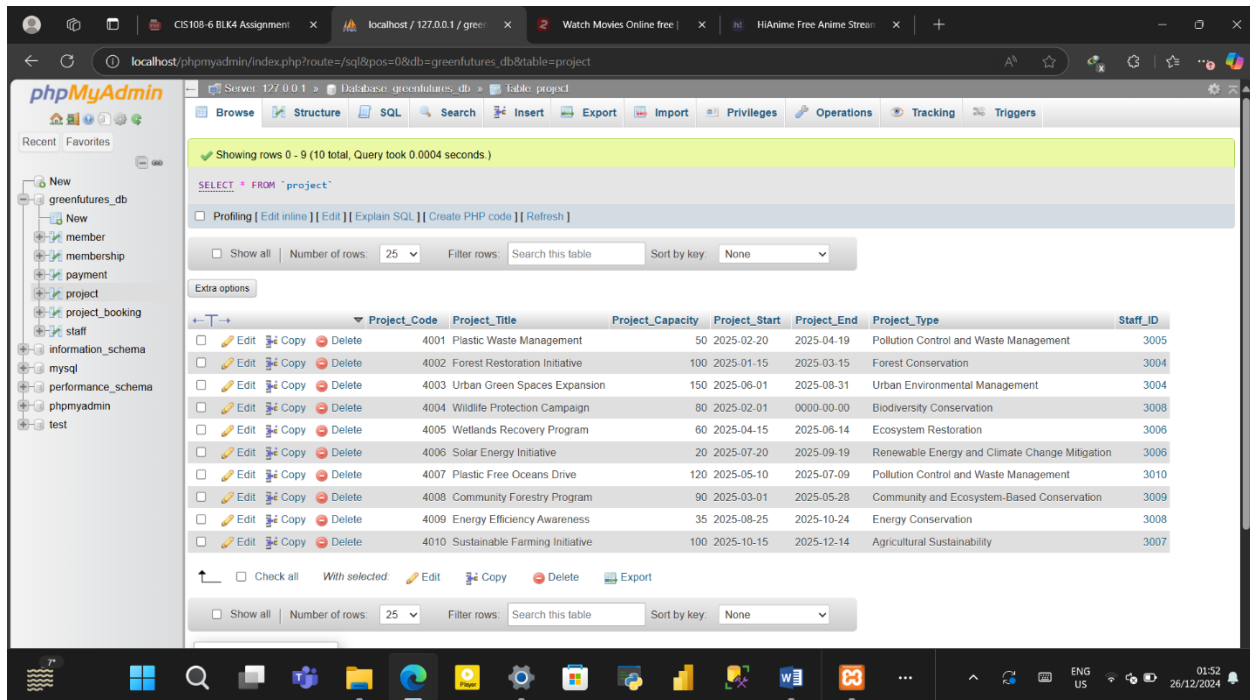


Figure 4.9.4.2: Data Successfully Entered into Project Table

4.9.5 Project_Booking Table Data Insertion

INSERT INTO project_booking
 (Project_BookingID, BookingDate, BookingTime, Staff_ID, Member_ID, Project_Code)
 VALUES

(5001, '2025-03-27', '12:34:09', 3006, 1003, 4003),
 (5002, '2025-04-02', '14:45:12', 3005, 1003, 4002),
 (5003, '2025-04-10', '09:30:45', 3007, 1004, 4003),
 (5004, '2025-04-17', '16:20:55', 3004, 1005, 4004),
 (5005, '2025-04-19', '11:05:33', 3005, 1006, 4005),
 (5006, '2025-04-24', '13:10:15', 3006, 1007, 4006),
 (5007, '2025-06-07', '08:25:50', 3008, 1008, 4007),
 (5008, '2025-06-19', '15:40:22', 3005, 1009, 4008),
 (5009, '2025-07-22', '10:50:17', 3004, 1010, 4009),
 (5010, '2025-08-10', '12:30:29', 3010, 1009, 4010),
 (5011, '2025-08-29', '14:45:00', 3007, 1002, 4004),
 (5012, '2025-08-30', '09:15:48', 3004, 1003, 4005),
 (5013, '2025-08-30', '11:50:25', 3009, 1004, 4006),
 (5014, '2023-08-30', '13:20:31', 3004, 1005, 4007),
 (5015, '2025-09-15', '16:35:45', 3005, 1006, 4008),
 (5016, '2025-09-16', '10:10:00', 3006, 1007, 4009),
 (5017, '2025-10-17', '08:55:15', 3009, 1008, 4010),
 (5018, '2025-10-18', '15:25:33', 3008, 1009, 4001),
 (5019, '2025-11-19', '12:05:10', 3008, 1010, 4002),

(5020, '2025-11-20', '14:50:47', 3010, 1002, 4003);

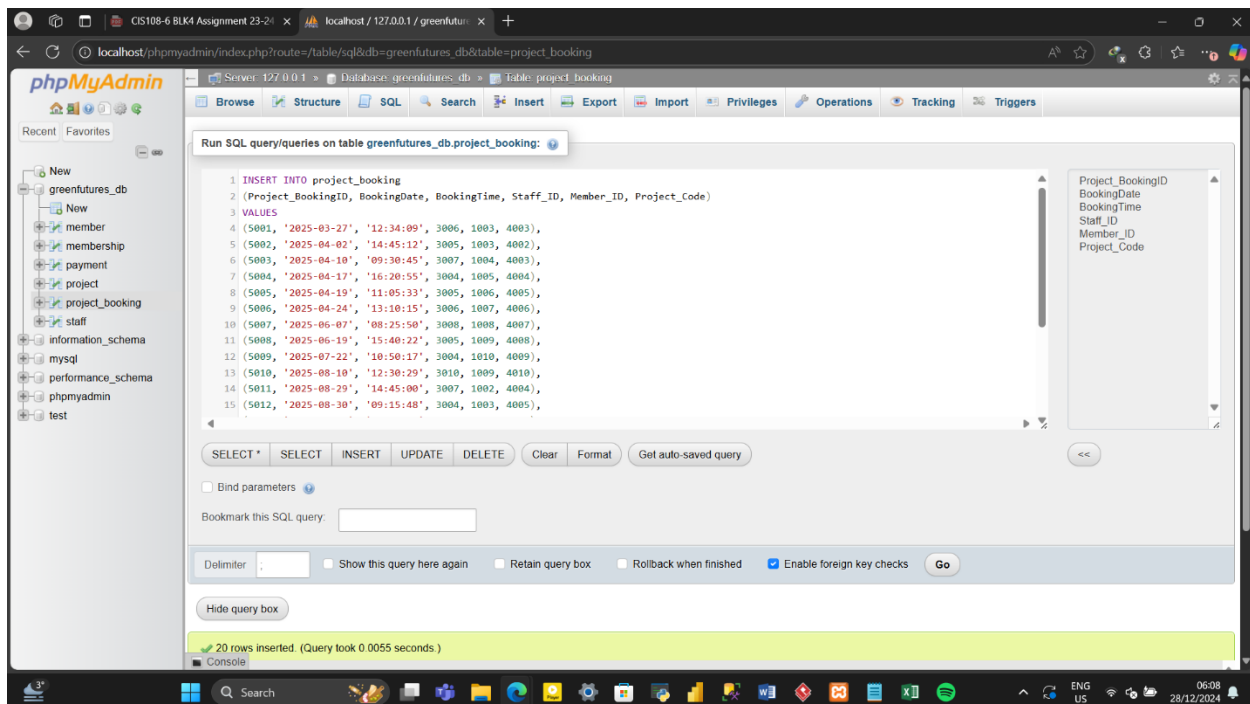


Figure 4.9.5.1: Data Entry for Project_Booking Table

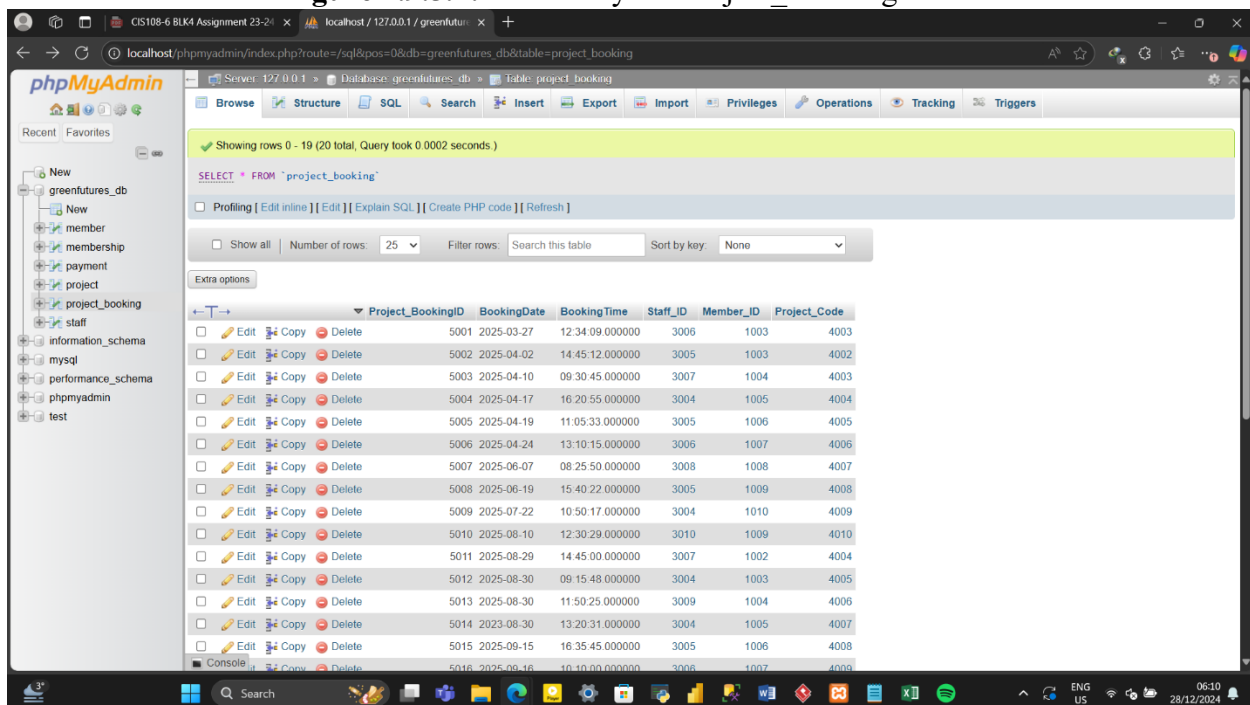


Figure 4.9.5.2: Data Successfully Entered into Project_Booking Table

4.9.6 Payment Table Data Insertion

INSERT INTO Payment

(Payment_ID, Payment_Date, Payment_Amount, Payment_Method, Member_ID)
VALUES

(6001, '2025-02-14', 40, 'Visa Card', 1004),
(6002, '2025-02-02', 70, 'MasterCard', 1002),
(6003, '2025-02-03', 100, 'PayPal', 1007),
(6004, '2025-03-04', 40, 'Bank Transfer', 1009),
(6005, '2025-03-05', 40, 'Visa Card', 1008),
(6006, '2025-03-06', 70, 'MasterCard', 1003),
(6007, '2025-03-07', 40, 'PayPal', 1009),
(6008, '2025-03-08', 100, 'Bank Transfer', 1001),
(6009, '2025-04-09', 70, 'Visa Card', 1005),
(6010, '2025-04-10', 70, 'MasterCard', 1006);

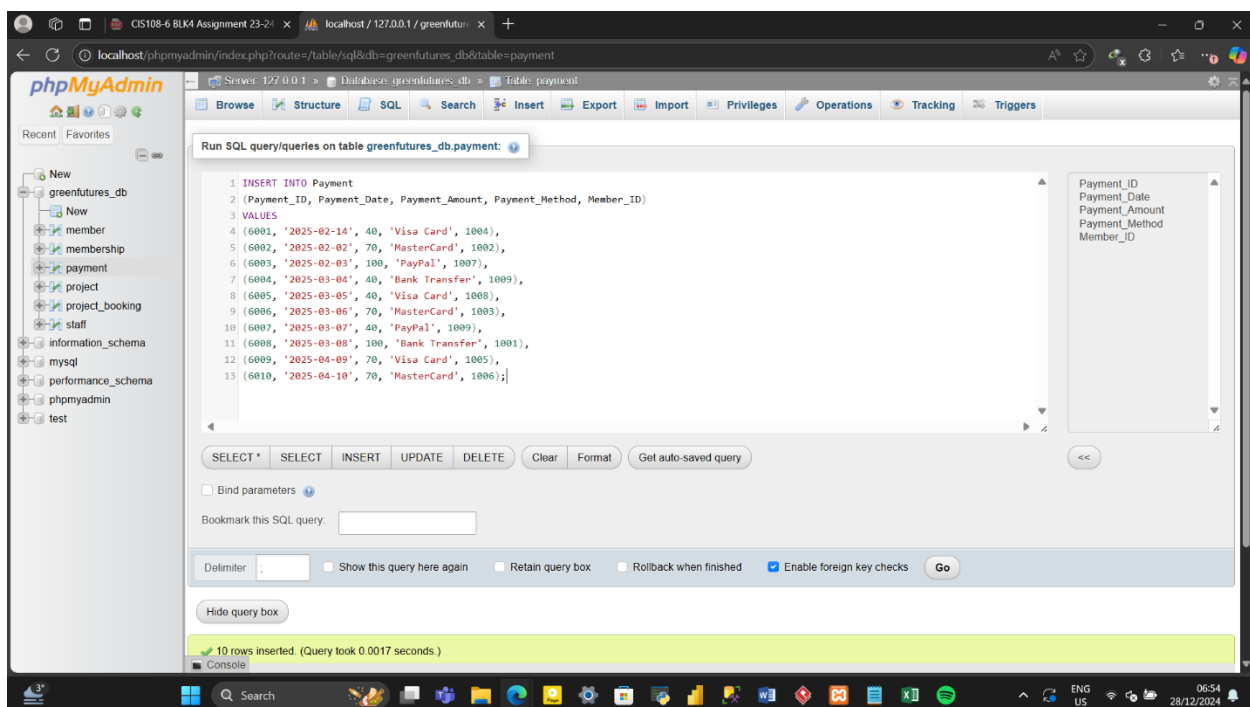


Figure 4.9.6.1: Data Entry for Payment Table

Showing rows 0 - 9 (10 total, Query took 0.0003 seconds)

```
SELECT * FROM `payment`
```

☐ Profiling [\[Edit inline \]](#) [\[Edit \]](#) [\[Explain SQL \]](#) [\[Create PHP code \]](#) [\[Refresh \]](#)

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

	Payment_ID	Payment_Date	Payment_Amount	Payment_Method	Member_ID
☐ Edit Copy Delete	6001	2025-02-14	40.00	Visa Card	1004
☐ Edit Copy Delete	6002	2025-02-02	70.00	MasterCard	1002
☐ Edit Copy Delete	6003	2025-02-03	100.00	PayPal	1007
☐ Edit Copy Delete	6004	2025-03-04	40.00	Bank Transfer	1009
☐ Edit Copy Delete	6005	2025-03-05	40.00	Visa Card	1008
☐ Edit Copy Delete	6006	2025-03-06	70.00	MasterCard	1003
☐ Edit Copy Delete	6007	2025-03-07	40.00	PayPal	1009
☐ Edit Copy Delete	6008	2025-03-08	100.00	Bank Transfer	1001
☐ Edit Copy Delete	6009	2025-04-09	70.00	Visa Card	1005
☐ Edit Copy Delete	6010	2025-04-10	70.00	MasterCard	1006

☐ Check all | With selected: [Edit](#) [Copy](#) [Delete](#) [Export](#)

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Query results operations

Console

Figure 4.9.6.2: Data Successfully Entered into Payment Table

Chapter 5

Testing and Deployment

5.0 Testing

Testing is a crucial process that checks the functionalities and correctness of the system as per the requirements of user. In this report, the testing of the developed system for Green Futures' Organization will be tested to check if the functional requirements set out by the user is satisfied or not. To test the system, SQL statements are used to query, retrieve and add data.

5.0.1: Test 1 - Insert a new member

Given we have a member table created, we can add by using the INSERT INTO command with the columns (Member_ID, FirstName, etc.) which match the structure of the Member table followed by the VLAUES command with contain details of the new member to be added. Thus,

INSERT INTO Member

(Member_ID, FirstName, Surname, DoB, Address, Email, Telephone, Gender, Nationality, MedicalCondition)

VALUES

(1012, 'Samuel', 'Johnson', '1990-05-15', '123 Elm Street, Springfield, SP1 3AB', 'samuel.johnson@gmail.com', 7894561230, 'M', 'United Kingdom', 'None');

Figure 5.0.1.1 and Figure 5.0.1.2 shows a visual representation of the commands to add a new member and the successful addition of a new member respectively.

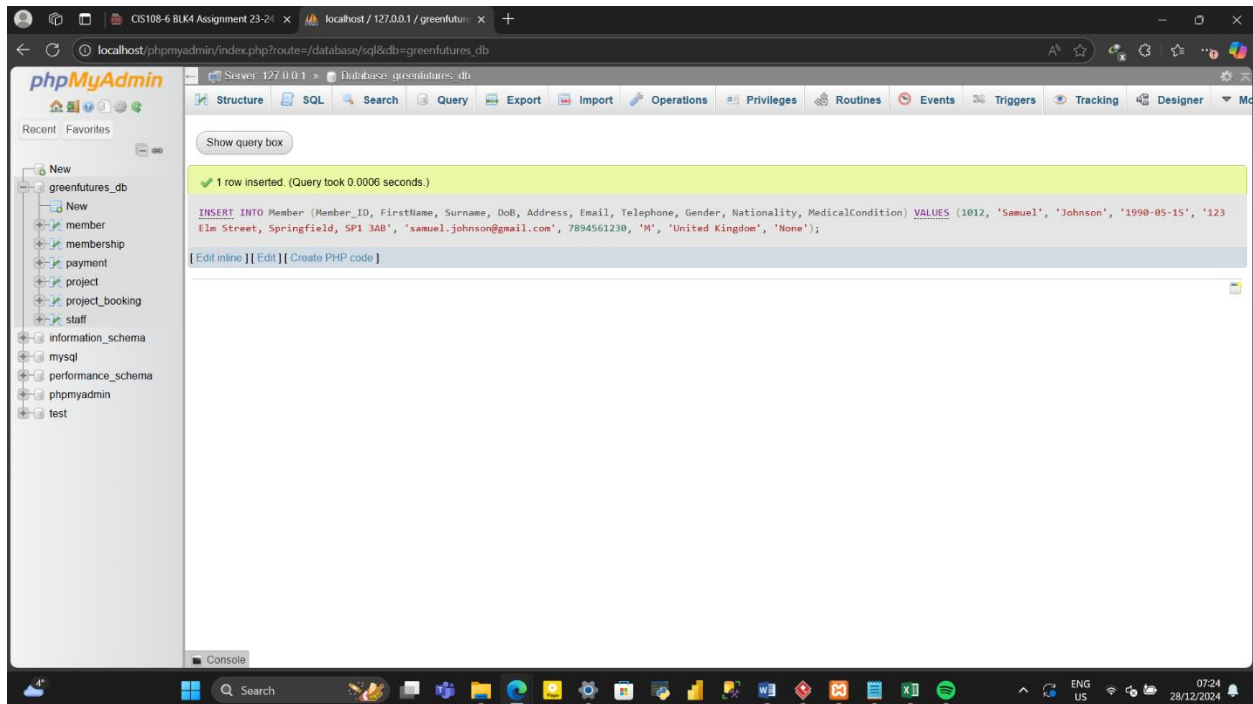


Figure 5.0.1.1: visual representation of the commands to add a new member

Member_ID	FirstName	Surname	DoB	Address	Email	Telephone	Gender	Nationality	MedicalCondition
1001	Imoh	Umoro	1994-04-25	72 Crosslands Stantonbury Road MK14 7AX	imoh.umoro@gmail.com	7324538856	M	Nigeria	Diabetes
1002	Aisha	Abdul	1990-09-12	45 Greenfield Road, Birmingham, B18 6NF	aisha.abdul@example.com	7894561230	F	Nigeria	Hypertension
1003	James	Owusu	1988-03-07	18 Kingsway, London, SW1E 6DD	james.owusu@example.com	7723456789	M	Ghana	Asthma
1004	Sophia	Musa	1995-11-22	89 Bridge Street, Manchester, M4 5DL	sophia.musa@example.com	7312345678	F	Nigeria	None
1005	Kwame	Asare	1985-01-15	32 Broad Street, Accra, Ghana	kwame.asare@example.com	233545678901	M	Ghana	Sickle Cell Anemia
1006	Emily	Smith	1992-06-10	24 Baker Street, Nottingham, NG1 6BQ	emily.smith@example.com	7789123456	F	United Kingdom	None
1007	Victor	Okonkwo	1989-08-30	50 Oakwood Avenue, Leeds, LS6 1BA	victor.okonkwo@example.com	7896543210	M	Nigeria	Epilepsy
1008	Amma	Nyarko	1993-04-19	12 Ridge Street, Kumasi, Ghana	amma.nyarko@example.com	233244567890	F	Ghana	Allergy
1009	Daniel	Brown	1987-12-05	78 High Street, Cardiff, CF10 1PX	daniel.brown@example.com	7890123456	M	United Kingdom	None
1010	Fatima	Yusuf	1996-02-11	60 Victoria Road, Abuja, Nigeria	fatima.yusuf@example.com	234706543210	F	Nigeria	Hypertension
1012	Samuel	Johnson	1990-05-15	123 Elm Street, Springfield, SP1 3AB	samuel.johnson@gmail.com	7894561230	M	United Kingdom	None

Figure 5.0.1.2: successful addition of a new member

5.0.2: Test 2 – Book a project

Project_Booking displays booked projects of members. Suppose a project needs to be booked for a member, this can be done by using the INSERT INTO Project_Booking and VALUES command and inserting the necessary information into their columns making sure the Staff_ID, Member_ID, and Project_Code exists in their respective tables. Thus,

INSERT INTO

Project_Booking (Project_BookingID, BookingDate, BookingTime, Staff_ID, Member_ID, Project_Code)

VALUES

(5022,'2023-12-01', '14:30:00', 3004, 1006, 4003);

Figure 5.0.2.1 and Figure 5.0.2.2 shows a visual representation of the commands to book a project and the successful booking of a project respectively.

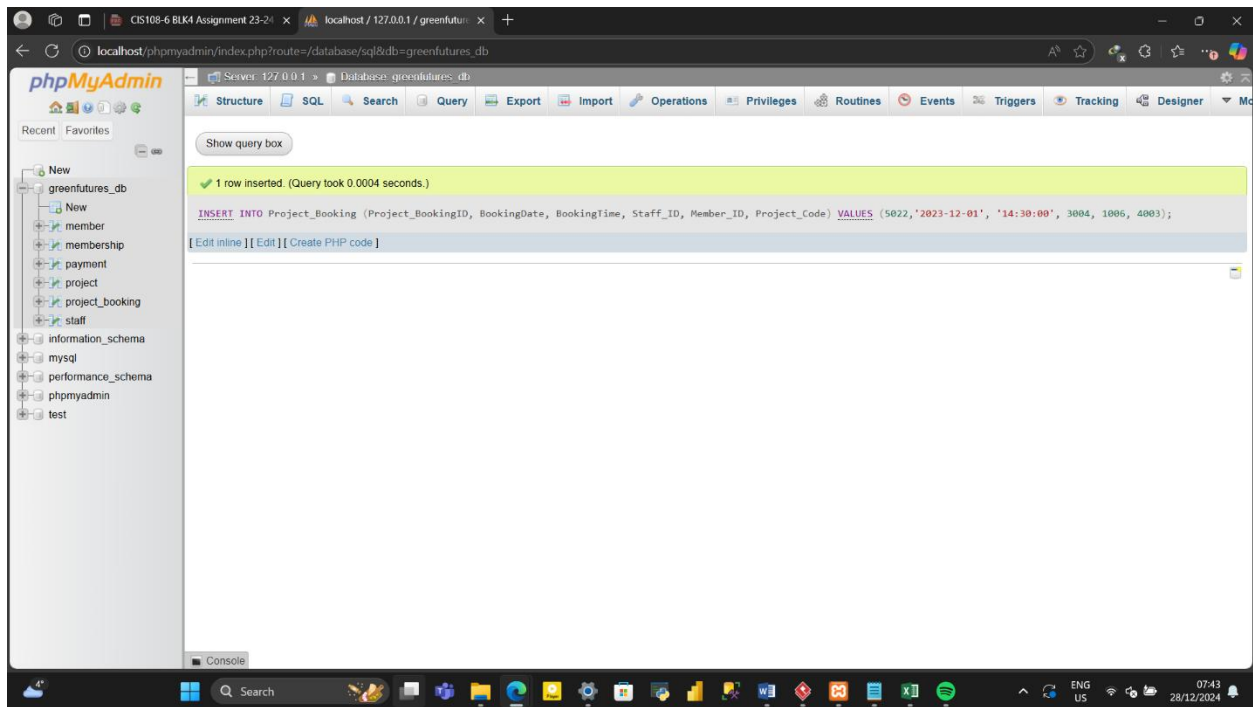


Figure 5.0.2.1: visual representation of the commands to book a project

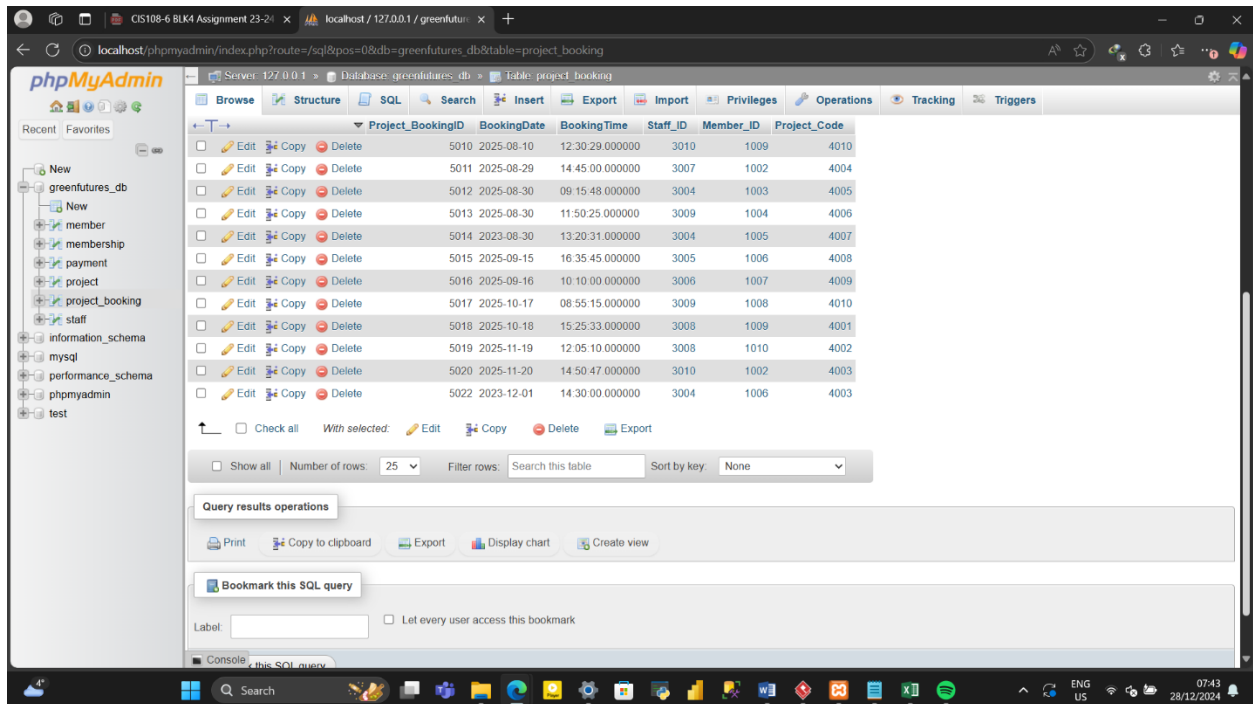


Figure 5.0.2.2: successful booking of a project

5.0.3: Test 3 – Check my project

To check the projects a member booked in a specific year, we can use the SELECT statement to return the fields from Member, Project and Project_Booking table. INNER JOIN to join the selected tables. The WHERE command to give specific conditions with regards the member we intend to check booked projects of and within the time frame selected. Thus;

```
SELECT member.FirstName,member.Surname,    Project.Project_Title,  
project_booking.Project_BookingID, project_booking.Member_ID,  
project_booking.Project_Code  
FROM project_booking  
INNER JOIN member ON project_booking.Member_ID=member.Member_ID  
INNER JOIN project ON project_booking.Project_Code=project.Project_Code  
WHERE member.Member_ID=1006  
AND project_booking.BookingDate BETWEEN '2025-01-01' AND '2025-12-31';
```

Figure 5.0.3.1 and Figure 5.0.3.2 shows a visual representation of the commands to check projects and the successful display of projects respectively.

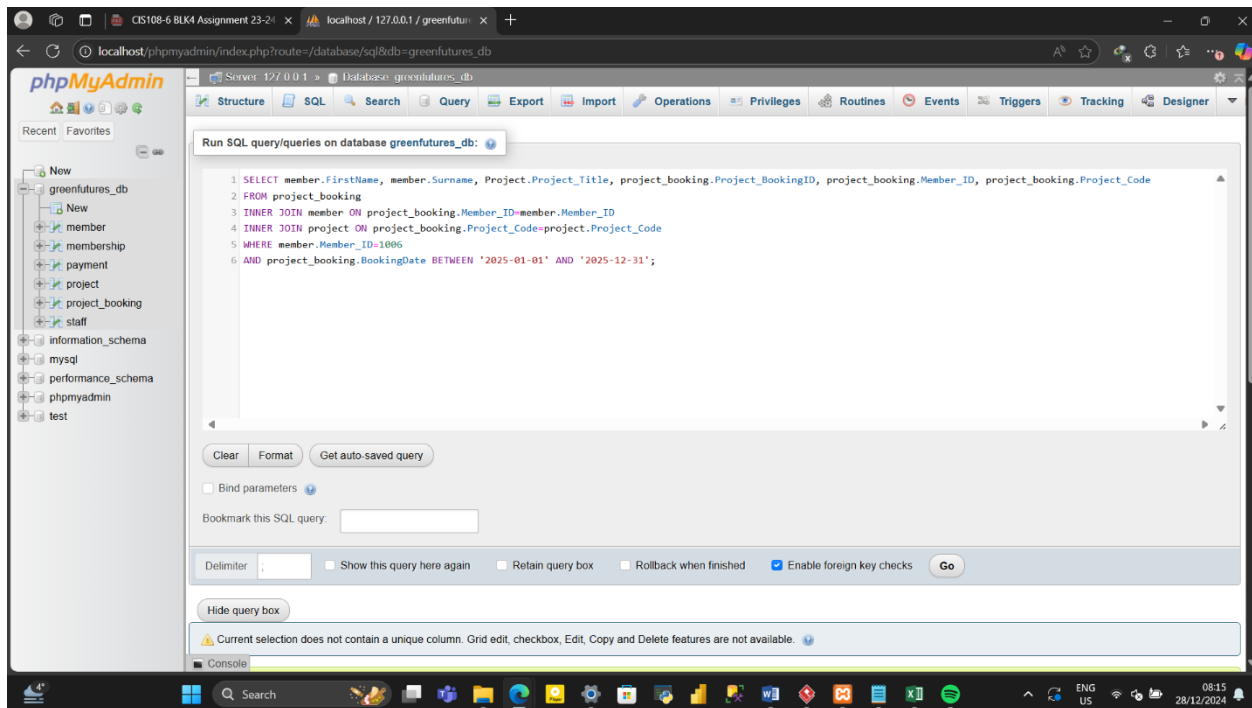


Figure 5.0.3.1: visual representation of the commands to check projects

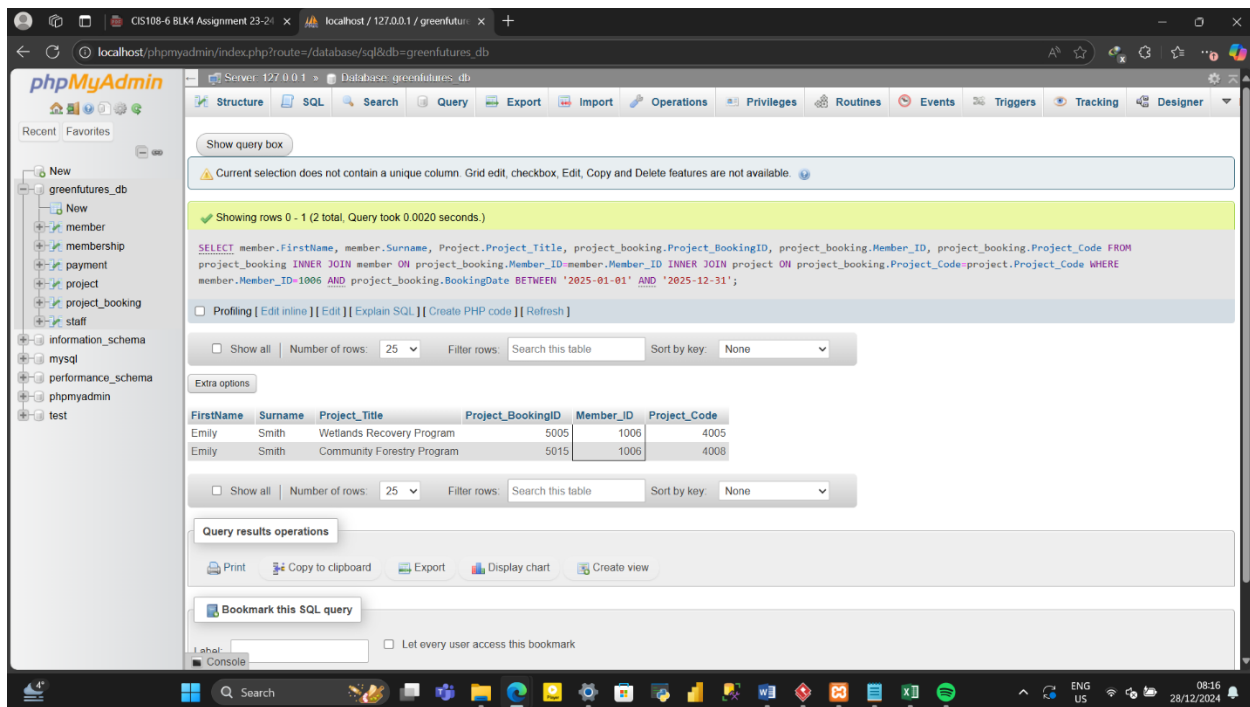


Figure 5.0.3.2: successful display of projects

5.1 System Deployment

The process of ensuring availability of a software system to its end users is critical for maintain user satisfaction and operational efficiency. Cloud deployment is a trusted setup to ensure the system remains accessible and reliable for end users by implementing load balancing and redundancy to handle hardware or network failures without affecting availability. With these types of system, cloud deployment is a better choice given its flexibility in using tools to monitor server health and performance to detect issues early, protection against cyber threats that could compromise sensitive information and unauthorized access, and regular data back up done to prevent loss of data due to hardware failure.

Chapter 6

6.0 Conclusion

In conclusion, the integration of a use case diagram, an activity diagram, and an entity-relationship database has proven instrumental in designing and implementing a comprehensive environmental conservation organization management system. The use case diagram provided a clear understanding of the system's functionalities and user interactions. It outlined various scenarios, from membership to project booking, and payment involved, enabling seamless user experience.

The activity diagram on the other hand, offered a detailed view of the system's process, illustrating the flow of activities from login / sign up stage to the booking confirmation or message display stage. This visual representation facilitated the identification of potential bottlenecks and allowed for optimizations to enhance efficiency and user satisfaction.

Lastly, the entity relationship database model served as the backbone of the system, organizing data into logical structures and establishing relationships between entities. This ensured data integrity, scalability, and ease of maintenance, crucial for the long-term success of Green Futures' Environmental Conservation Organization management.

Overall, the combination of these methodologies has resulted in a robust and user-friendly environmental conservation organization management system, capable of meeting the diverse needs of both staff and members. Through careful planning and implementation, the database-driven approach has laid a solid foundation for future enhancements and advancements in environmental conservation organization management.

6.1 Limitations and Recommendations

Due to limited time allocated for this short project, each and every testing done before making a software system available has not been done to actually confirm what is believed to be functioning properly. The software system also contains only a few handfuls of information due to limited time to input large amount of information.

With enough time in the near future, series of information could be entered while testing all aspect of the system to confirm and be sure of the systems functionalities operating and working as intended.

Reference

Mark, S., Lynne, M., and Beath, C. M. (1995) 'The Information Technology Interaction Model: A Foundation for the MBA Core Course', Special Issue on IS Curricula and Pedagogy (Sep., 1995), *MIS Quarterly*, 19(3), pp. 361-390.

Coronel, C. and Morris, S. (2016) *Database Systems: Design, Implementation, and Management*. 12th ed. Boston, USA: Cengage Learning.

Connolly, T.M. and Begg, C.E., 2005. *Database systems: a practical approach to design, implementation, and management*. Pearson Education.

F. Hayat, A. U. Rehman. K. S. Arif, K. Wahab and M. Abbas, "The Influence of Agile Methodology (Scrum) on Software Project Management," 2019 20th IEEE/ACIS International Conference on Software Engineering, Artificial Intelligence, Networking and Parallel/Distributed Computing (SNPD), Toyama, Japan, 2019, pp. 145-149, doi: 10.1109/SNPD.2019.8935813.

Coronel, C. and Morris, S. (2016) *Database System: Design, Implementation, and Management*. 12th ed. Boston, USA: Cengage Learning.